

编译原理专题实践

周晓宇

目录

- [课程主要内容](#)
- [课程主要目的](#)
- [基本流程](#)
- [Lex部分主要工作](#)
- [Yacc部分主要工作](#)
- [中间代码生成](#)
- [分工建议和参考进度](#)
- [参考资料](#)

目录

- 实验报告与答辩
- 理论课内容
- 期末提交内容
- Lex输入文件格式解析
 - Lex Source Definitions
 - Lex Actions
 - Ambiguous Source Rules
- Lex输入文件处理的基本流程
 - 扩展的RE转普通RE
 - 把扩展的RE直接转换成NFA
- 算符优先分析法

目录

- Yacc
 - The Definitions Section of Yacc
 - The Rules Section of Yacc
 - Actions
- 正规式到NFA的转换
- 多个NFA的合并
- 确定化与最小化

目录

- [语法分析](#)基本流程
- [LALR \(lookahead-LR\) Parsing](#)
- [语义分析](#)
 - [声明语句的处理](#)
 - [LLVM IR](#)
 - [LLVM IR的形式](#)
 - [编写自定义的 LLVM IR 生成器](#)
 - [中间表示: Jimple](#)
 - [References](#)
 - [Jimple的语法成分](#)
 - [APIs to create Jimple Codes](#)
 - [Jimple 代码生成](#)

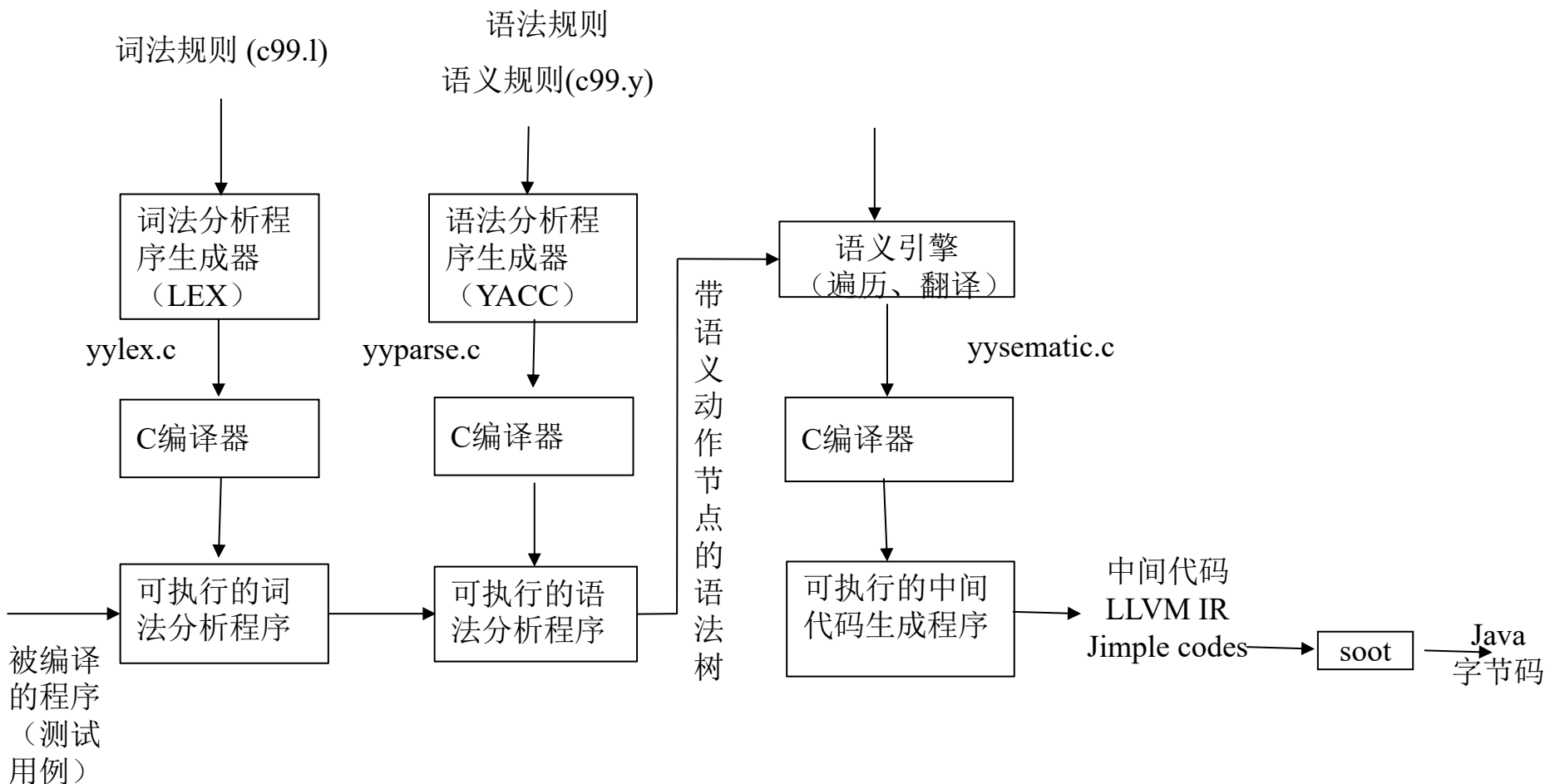
课程主要内容

- 通过自己所设计的工具生成一种程序设计语言（或其子集）的词法和语法分析器
 - Lex
 - Yacc
- 作为测试对象的语言不限（授课以c99为例）
- 实现工具时所使用的语言不限
- 欢迎使用AI辅助编程，请在报告中记录所使用的开发平台、AI工具，记录完整的提示和验证过程。

课程主要目的

- Lex和Yacc都有工具可用，为什么我们还要自己编写？
 - 课程设计的目的不是创新，是训练。
 - 加深对编译原理及编译程序构造过程的理解
 - 增强程序设计能力
 - 学会编写系统软件
 - 为《计算机系统综合课程设计》奠定基础

基本流程



Lex部分主要工作

- SeuLex的基本流程
 - Lex输入文件的解析 (Lex RE $\Rightarrow$ 常规RE)
 - 常规RE的解析 (转语法树或后缀表达)
 - 将每个正规式转换到一个NFA
 - 将所有正规式对应的NFA合并为一个大的NFA
 - NFA的确定化
 - DFA的最小化
 - 由最小DFA生成词法分析程序
 - 测试

Yacc部分主要工作

- SeuYacc
 - Yacc输入文件的解析
 - 上下文无关文法到对应LR(1)文法的下推自动机的构造
 - 求First集
 - item的扩展及预测符的计算
 - 变迁关系的构造。
 - LR(1)文法的下推自动机到相应分析表的构造
 - LR(1)总控程序的构造（查表程序）
 - LR(1)到LALR(1)的转换

中间代码生成

- Lex、Yacc部分针对全集，中间代码部分针对子集（Lex和Yacc的输入文件再做一个子集版）
- 语义分析的输入
 - 句柄序列（相应的产生式序列），或
 - 语法树
- 语义分析
 - 生成语法树数据结构及可视化
 - 将语义动作嵌入语法树
 - 划分基本块（为生成LLVM IR服务）
 - 生成中间代码
 - 执行语义动作生成中间代码

中间代码生成

- 中间代码形式
 - 按照书本生成三地址语句或四元式
 - 三地址指令中没有变量声明部分，需要生成符号表
 - LLVM IR (c++, linux)
 - 有变量声明
 - Jimple(Java, windows)
 - 有变量声明
- 对C语言的子集构造语义规则
 - 构造带语义节点的语法树
 - 构造符号表
 - 生成三地址语句
 - 声明和定义
 - 语句
 - 函数调用语句
 - 其他语句

中间代码生成

- 将语法规则改写为translation scheme
 - 实现Lex的同学早期可以选择跳过语义动作节点
 - 以 { } 或其他符号定界
- 设计标记语法树的数据结构
 - 为语法节点增加语义信息域
- 首先生成带语义动作节点的语法树
- 先序遍历语法树执行语义动作。
 - 生成相应的c程序
 - 该程序的执行再生成中间代码

中间代码生成

- 区分语法分析阶段执行的动作和中间代码生成阶段执行的动作
 - 编译时：构造语法树、构造符号表
 - 运行时：实现动态语义：加减赋值等
- 按照开源编译工具的格式生成中间表示
 - LLVM IR (Linux C++)
 - Jimple (Java, 经soot转换为Java字节码)
- 有参考，先做一点点，然后逐步扩展

分工建议和参考进度

- Lex、Yacc、中间代码生成各一人
- 参考进度
 - 第1周：
 - 组队、分工
 - 确定Lex输入文件
 - 设计Lex输入文件解析的算法和数据结构，给出伪代码
 - 给出Lex部分进度计划（每周实现哪些关键算法）
 - 开发语言、平台的确定
 - Gitee等联合开发平台的熟悉和选用
 - AI辅助编程平台的选择和学习

- 学习AI辅助开发技术

- Trae等AI辅助编程平台的教程和开源资料

- 斯坦福大学的AI辅助编程课程：

- <https://themodernsoftware.dev>

- *Claude*构建的C编译器：

- https://mp.weixin.qq.com/s/NYKDLw_yVOJEvrobNLfGgg

- <https://github.com/anthropics/claudes-c-compiler>

参考进度

– 第2周：

- 确定Yacc输入文件
- 设计Yacc输入文件解析的算法和数据结构，给出伪代码
- 确定各人的接口方式、确定主要环节的数据结构
- 初步给出输出语法树的语义规则
- 说明和定义部分的语义规则
- 符号表构造的语义规则
- 确定语法分析部分的进度计划（每周实现哪些关键算法）

– 第3周：

- 初步设计程序架构、函数调用关系
- 语句部分的语义规则
- 语义规则向translation scheme的转换

参考进度

– 第4周：

- Lex、Yacc详细设计及详细设计文档、主要算法伪代码。
- 语义规则实现（用C++或Java）方案
- 语义部分的详细进度计划

– 第5-9周

- 具体实现
- 第5周末向助教老师提交详细设计方案作为中期检查。
- 第5周实现：
 - Lex输入文件的解析（Lex RE $\Rightarrow$ 常规RE）
 - Yacc
 - » 输入文件的解析。
 - » 求非终结符的First集
 - 中间代码生成
 - » 语义规则的读入

参考进度

- 第6周实现
 - Lex
 - » 正规式的解析（转语法树或后缀表达）
 - » 将每个正规式转换到一个NFA，NFA的可视化
 - » 将所有正规式对应的NFA合并为一个大的NFA
 - Yacc
 - » Item闭包的构造
 - » 预测符的计算
 - 中间代码生成
 - » 带语义动作节点的语法树的生成
 - » 调试：手工构造一个语法树，确定规约顺序

参考进度

- 第7周实现
 - Lex
 - » NFA的确定化，DFA可视化
 - Yacc
 - » LR分析状态变迁的构造
 - 中间代码生成
 - » 先序访问带语义动作的语法树，编译时语义动作的实现（C++或Java）
- 第8周实现
 - Lex
 - » DFA最小化
 - » 根据DFA进行此法分析
 - Yacc
 - » LR分析表的构造和总控程序的构造
 - » LALR分析的实现
 - 中间代码生成
 - » 运行时语义动作的实现（C++或Java）

参考进度

- 第9周实现
 - Lex、Yacc、中间代码生成联调、测试
 - 测试、课程设计报告撰写、ppt准备。
- 第10-12周
 - 验收答辩

参考资料

- M.E.LeskandE.Schmidt. Lex – A Lexical Analyzer Generator.
 - (简洁)
- John R. Levine, Tony Mason.著. 杨作梅, 张旭东等译.Lex与Yacc. 机械工业出版社. 2003.
 - (相关工具不易获取, 细节含义难以确认)
- John levine著.陆军译.flex与bison.东南大学出版社.2011.
 - (相关工具易于获取, 可以通过运行现有的解析器确认细节含义)

参考资料

- 徐宝文.实用C语言详解.1996.
- ISO C 1999 Definition. ISO/IEC 9899:1999
- Rationale for International Standard—
Programming Languages— C: <https://open-std.org/JTC1/SC22/WG14/www/C99RationaleV5.10.pdf>
- 群文件中的其它资料

参考资料

- Flex和Bison

- <https://github.com/westes/flex>
- <https://www.gnu.org/software/bison/>

- CSDN上的相关帖子

- 基于C语言设计符号表

https://blog.csdn.net/newlw/article/details/126839728?ops_request_misc=&request_id=&biz_id=102&utm_term=%E5%9B%9B%E5%85%83%E5%BC%8F%20cmi nus.y&utm_medium=distribute.pc_search_result.none-task-blog-2~all~sobaiduweb~default-0-126839728.142^v99^pc_search_result_base4&spm=1018.2226.3001.4449

参考资料

– 编译原理实验

https://blog.csdn.net/qq_45795586/article/details/122592648?ops_request_misc=%257B%2522request%255Fid%2522%253A%2522170927807716800227488798%2522%252C%2522scm%2522%253A%252220140713.130102334.pc%255Fall.%2522%257D&request_id=170927807716800227488798&biz_id=0&utm_medium=distribute.pc_search_result.none-task-blog-2~all~first_rank_ecpm_v1~rank_v31_ecpm-1-122592648-null-null.142^v99^pc_search_result_base4&utm_term=cminus.y%20%E5%9B%9B%E5%85%83%E5%BC%8F&spm=1018.2226.3001.4449

参考资料

- LLVM编译器实战教程.机械工业出版社.[第1章 编译和安装LLVM — Getting Started with LLVM Core Libraries 文档 \(getting-started-with-llvm-core-libraries-zh-cn.readthedocs.io\)](#)
- Getting Started with LLVM Core Libraries:
<https://faculty.sist.shanghaitech.edu.cn/faculty/son-gfu/course/spring2018/CS131/llvm.pdf>.
- Mayur Pandey Suyog Sarda 著,王欢明译. LLVM Cookbook中文版.北京: 电子工业出版社, 2016.6

参考资料

- Raja Vallee-Rai. SOOT: A Java Bytecode Optimization Framework. (Thesis) McGill University.
 - 第5章(API)
- Raja Vallee-Rai, Laurie J. Hendren. Jimple: Simplifying Java Bytecode for Analyses and Transformations.
 - 内含部分Jimple grammar (page 6 and 7)。
- Raja Vallee-Rai. The Jimple Framework. Sable Technical Report No. 1. 1998
- Jimple grammar:
<http://releases.strategoxt.org/docs/syntax/jimple-front/stable/docs/html/languages/jimple/Main.sdf.html>

参考资料

- Jimple Examples.

<https://soot-oss.github.io/SootUp/jimple/#jimple-grammar-structure>

- Jimple API

- <https://javadoc.io/doc/ca.mcgill.sable/soot/latest/soot/jimple/Jimple.html>

- <https://www.sable.mcgill.ca/soot/doc/soot/jimple/Jimple.html>

- Tutorials for soot.

<https://github.com/soot-oss/soot/wiki/Tutorials>

- 群文件 “编译原理实验报告(Jimple)”

参考资料

- 冯燕等著.编译原理课程设计. 浙江大学出版社.2007.
- 王雷,刘志成,周晶.编译原理课程设计.机械工业出版社.2005.

实验报告与答辩

- 不超过3人一组
- 实验报告
 - Lex、Yacc、中间代码生成分开写
 - 详细、完整
 - 目标语言描述（Lex和Yacc输入文件）
 - 程序架构与接口
 - 具体分工协作
 - 实际工作流程和进度
 - 数据结构
 - 相关讨论、实际效果（性能分析）
 - 主要算法的伪代码
 - 特别是书本上没有的部分
 - 使用说明
 - 测试报告

实验报告与答辩

- AI辅助的交互过程
 - 提示过程
 - 每一步的结果及对结果的验证、分析、手工调整
- 突出特色（重点突出，可辟专门章节详细介绍自己的特色部分）

实验报告与答辩

- 答辩
 - 答辩前把实验报告发给教师
 - 每个人报告自己的部分
 - 外加组内互评作为参考
 - 每个同学发邮件给我，为组内的每个人评分（含自己）
 - 迅速、清晰地回答问题（每人约7-8分钟，含lex、yacc、中间代码生成）
 - 要求概念清晰、代码熟悉

理论课内容

- Lex的输入格式（扩展的RE）解读
 - c99.1
- Yacc的输入格式解读
 - C99.y
- LALR分析
- 词法分析和语法分析的数据结构及程序架构
- 简单优先分析法和算符优先分析法
- 符号表的结构
- 声明语句的处理
- Jimple语言
- 报告撰写（示例）

期末提交内容

- 全部电子文档提交给课代表
- 以全组同学学号+姓名命名文件夹
- 内容（文件夹）：
 - Lex和Yacc的输入文件（含语义规则和translation scheme）
 - 词法、语法分析程序生成器的源程序
 - 中间代码生成模块的源程序
 - 生成的词法、语法分析程序源程序
 - 词法分析、语法分析的测试用例和测试结果
 - 生成的中间代码
 - 实验报告及ppt

Lex

- Lex输入文件（Lex source）的基本格式：

```
{definitions}
```

```
%%
```

```
{rules}
```

```
%%
```

```
{user subroutines}
```

Lex输入文件中的Definitions

- 包含在%{ 和}%之间的行中的内容都被复制出去。
 - 见c99.1。
- 用于Lex的regular definitions在第一个%%分隔符之前给出。
 - 从第1列开始。
 - 每行的格式如下：

name translation
 - 见c99.1
 - Regular definition, 没有action
 - 对所有的rules可见。

Lex输入文件中的Regular Expressions

- 字母表（Alphabet）
 - 目标语言的字符集（ text characters ）
 - Lex中扩展的正规式的operator（operator characters , or meta language symbol）
- Lex中的操作符（ Operators）
 - " \ [] ^ - ? . * + | () \$ / { } % < >
 - 如果这些符号要被用作目标语言的符号，需要通过转义（ an escape should be used） 。

被Regular Definitions定义的名字: {}

- {digit}代表在regular definition中已经被定义的名字 (name) digit.
 - 见c99.1。

几个普通的C转义

- `\n`: 换行 (newline) ;
 - `\t`: tab,
 - `\b`: backspace.
 - `\\`: 代表\本身。
-
- C语言的转义和Lex中的转义可能不完全相同

操作符：[]

- 外延式定义符号集合 (*character classes*)
 - [abc]等价于集合{a b c}。

‘[’, ‘]’中的‘-’

- 用边界定义集合
 - -左右的符号表示边界（ ranges ）。
 - alpha [A-Za-z]
 - digit [0-9]
 - -左右的符号表示range的边界
- 如果希望在字符类中包含字符 ‘-’ （仅表示它自己），则它应该是第一个或最后一个（没有边界）；
 - [-+0-9]表示 ‘-’ 、 ‘+’ 和个位十进制数字。

‘[’ ‘]’中的‘^’

- 用补集定义集合
 - ‘^’ 与 ‘[’ ‘]’ 对组合应用
 - ‘^’ 必须位于 ‘[’ ‘]’ 对中的首位;
 - 表示补集。
 - $[\text{^}abc]$ 匹配除了 a 、 b 、 c 之外的所有符号。
 - $[\text{^}a-zA-Z]$ 匹配除了字符以外的所有符号。

通配符‘.’

- 除了换行符以外的任意符号。

引号： " "

- “ ” 括起来的都被当做是目标语言的符号
 - 见山就是山，见水就是水
 - 主要用于表达和meta-language operator长得一样的 text characters.
 - + 是一个元语言操作符。
 - xyz“++” 表示目标语言中的字符串 xyz++ 。
 - “xyz++” 作用同上。
 - xyz\+\+ 作用同上。
- 在表达式中放入空格符。
 - "x yz++"

表达式的重复

- a^*
- a^+
- $[a-z]^+$
- $[A-Za-z][A-Za-z0-9]^*$
- $a\{1,5\}$ 匹配 连续出现1-5次的a。
 - a_1^5

操作符？

- 0个或1个。
- ？ 的操作数位于？的左侧。
- $ab?c$ 匹配 ac 或 abc 。

选择和打包

- 选择：|
- 打包：由()形成的整体
- $(ab \mid cd)$
- $(ab \mid cd^+)?(ef)^*$

Lex输入文件处理的基本流程

- 根据%%区分三个部分
 - 简单的字符串匹配
 - 对不同部分进行不同的处理
 - 对规则部分，读入RE-ACTION对。

扩展的RE的处理流程

- 扩展的RE中操作符的分类
 - 先确定操作数
 - {}
 - 通配符: .
 - []以及其中的^和-
 - 转义操作
 - 对连接、选择、闭包操作的扩展（正规式的operator）
 - 双引号、+、?、a{1,5}
 - C99.1中没用到:
 - \$ < >
 - / (未在RE中作为operator，但是在目标语言中出现，作为注释/*...*/的定界符出现。)

扩展的RE的处理流程

- 处理顺序
 - 先具体化操作数，后进行操作。
 - 针对第二类操作符，可以有两种处理方式
 - 扩展的RE转普通RE、中缀转后缀、后缀转NFA
 - 针对第二类操作符，分别制定直接转成NFA的规则，通过算符优先文法建立语法树，遍历语法树生成NFA

扩展的RE转普通RE

1. 正规定义的处理，将{ }内的 id 替换为被定义的内容。
 - 从前向后处理
2. 处理通配符 . 。
3. 处理[]内内容。
 1. 处理 - 。
 2. 处理 ^ 。
 3. 添加 | 和括号。
 - [xyz] -> (x|y|z)。
4. 处理双引号
 - 出现在双引号中的操作符需要在其前面补充转义符\;
5. 处理可选操作符? 和正闭包+、处理花括号中的重复次数。

- 添加连接符（为中缀转后缀做准备，可以采用符号-）。
 - 当前是普通字符，并且前面的字符不是（和|
 - 当前是[或者(,并且前面的字符不是（和|

扩展的RE的处理流程

- 在进行具体转换过程中，使用一系列变量来记录当前表达式所处的状态（是否处在括号中，是否处于转义状态，是否处在引号中等）。
- 检查各操作符是否被正确使用，包括：各种括号（（）、{}、[]、<>、`"）是否配对；符号出现的位置是否正确（如*、+、？符号前不能为空；/、^等符号只能出现在指定位置；|符号前后均不能为空）；花括号中的内容是否为正确；引用的正则定义是否存在等。
- 检查是否有嵌套
 - {}、[]、`"

Lex中的动作 (Actions)

- 接受状态的动作。
- `{rules} -> {regular expressions} {actions}`
- 如果只是想忽略被匹配的字符串，并不想有任何具体动作：
 - 用C语言的空语句 (null statement) “;” 作为动作
 - `[ \t\n] ;` 忽略blank, tab, and newline
- 避免重复书写相同的动作
 - | | |
|-------------------|---|
| <code>" "</code> | |
| <code>"\t"</code> | |
| <code>"\n"</code> | ; |

效果同上。

输出被匹配的字符串

- Lex把被匹配的字符串放在名为yytext的external character array。
 - [a-z]+ printf("%s", yytext);
 - [a-z]+ ECHO; //效果同上，即原样输出yytext中的string。
 - 见 C99.1中的Count() .

被匹配字符串的长度

- `yyleng`:
- `[a-zA-Z]+ {words++; chars += yyleng;}`
 - 计数输入中词和字符的数量。
- `yytext[yyleng-1]`
 - 数组下标从0开始。
 - 表示被匹配字符串中的最后一个字符。

上下文敏感（ Context sensitivity ）的匹配

- 如果RE的最后一个符号是\$，该表达式只匹配行末的字符串。
 - $ab\$$ 等价于 $ab\wedge n$
- 表达式 ab/cd 匹配字符串 ab ，条件是 ab 后面紧跟着的是 cd 。。

输出部分被匹配的字符串

- 调用 `yyless (n)` 仅保留被匹配字符串的前 `n` 个字符。
 - $n \leq \text{yyleng}$

输出部分被匹配的字符串

```
==-[a-zA-Z] {  
    printf("Op (==) ambiguous\n");  
    yless(yyvaleng-1);  
    ... action for == ...  
}
```

把 “**==**” 作为一个operator，等价于 *==-[A-Za-z]* （三个字符取前两个）

```
==-[a-zA-Z] {  
    printf("Op (==) ambiguous\n");  
    yless(yyvaleng-2);  
    ... action for = ...  
}
```

把 “**=**” 作为一个operator，等价于 *=/[A-Za-z]* （三个字符取第一个。
在特定上下文内取字符）

Lex遇到输入文件结束

- 函数yywrap返回1表示正常结束。
- yywrap返回0，Lex继续读取下一输入文件。
- yywrap缺省返回1。
- 见 c99.1.

二义性处理规则

- 具有二义性的specification

- 1) 最长匹配原则

- 规则

integer keyword action ...;

[a-z]⁺ identifier action ...;

integers : 识别为an identifier

最长匹配

- `'.*'`
 - 输入: `'first' quoted string here, 'second' here`
 - 匹配: `'first' quoted string here, 'second'` (以单引号结束)
- `'[^\n]*'`
 - 输入: `'first' quoted string here, 'second' here`
 - 匹配: `'first'` (消除了不确定性)

最长匹配

%%

-?[0-9]+

```
int k;
```

```
{//识别正负整数
```

```
k = atoi(yytext); //字符串转整型数
```

```
printf("%d",
```

```
k%7 == 0 ? k+3 : k);
```

```
}
```

-?[0-9.]+

```
ECHO; //识别正负小数，更长的  
//匹配
```

最长匹配原则

- 2) 如果有多个规则匹配同样长度的字符串，则排在前面的优先。

Lex.1

- C99 Lex/Flex & YACC/Bison Grammars:

<https://gist.github.com/codebrainz/2933703>

- c99.1
- Lex输入文件解析热身：
 - Exercise 3.3.7-3.3.11
- 关键字识别的效率问题
 - Exercise 3.4.3-3.4.12

正规式到NFA的转换

- 正规表达式的规范化
- 中缀形式的正规式到后缀形式的正规表达式转换
 - 二叉树的前序、中序、后序遍历和表达式的前缀、中缀、后缀表达
 - <https://www.cnblogs.com/bigsai/p/11393609.html>
 - <https://blog.csdn.net/qiantujava/article/details/17009653>
 - Bottom-up地组装，生成每个正规式的NFA
 - $(a|b)c \Rightarrow ab|c \cdot$

把扩展的RE直接转换成NFA

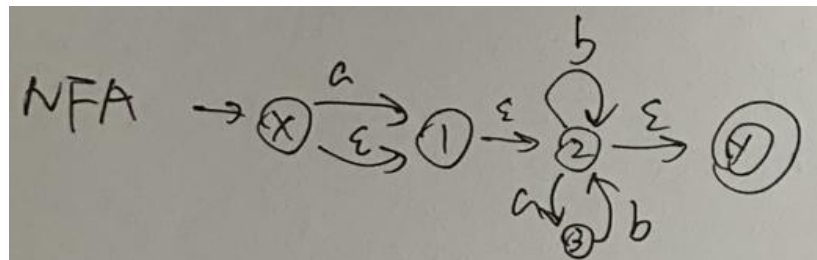
- 也有同学直接把扩展的RE转换成NFA。
 - 参考习题5.2.6
 - 通过语义规则把常规RE转换成NFA
 - 根据扩展RE的operator的语义设计相应的生成NFA的函数。例如
 - dot: .运算符
 - 初始状态添加任意一个字符都能转移到同一个状态，并将之设置为终止状态
 - dash: -运算符
 - 初始状态添加from到to中的任意一个字符都能转移到同一个状态，并将之设置为终止状态
 - square: []运算符
 - 初始状态添加[]中的任意一个字符都能转移到同一个状态，并将之设置为终止状态

- not: ^运算符
 - 初始状态添加任意不为target集合中的任意一个字符都能转移到同一个状态，并将之设置为终止状态
- repeat: {}运算符
 - 将max个nfa串起来，然后将前max-min个nfa用epsilon表达式连接到accept状态
- 各种运算符的协调问题、运算符与运算数的匹配问题。
 - 使用算符优先文法处理。
 - 参考优先级

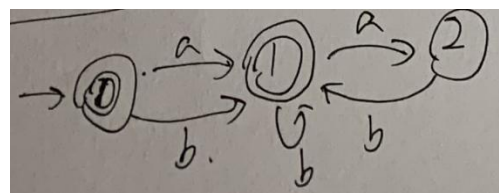
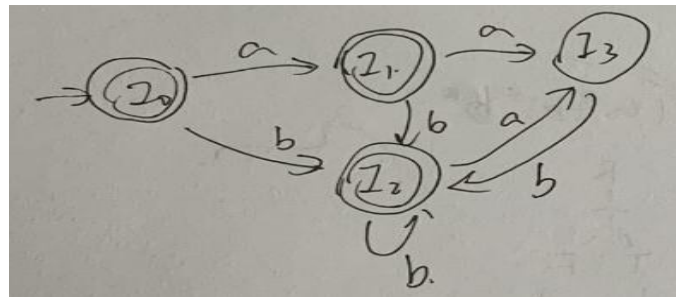
运算符		^	+	*	?	{	\$	(
栈内优先级	2	4	6	6	6	6	0	0
栈外优先级	1	3	5	5	5	5	-1	11

确定化与最小化

- 确定化（略）
- 最小化（略）。
 - 初始划分：根据action的不同将接受状态划分开。
 - 可以使用error状态。
 - 有进无出的死状态。
- 例子
 - $(a|\epsilon)(b|ab)^*$
 - 确定化：.....
 - 第三步：最小化。
 - 初始化：接受态集合（区分不同action的接受态）和非接受态集合
 - 求异亦求同。
 - 第四步：去掉error状态。

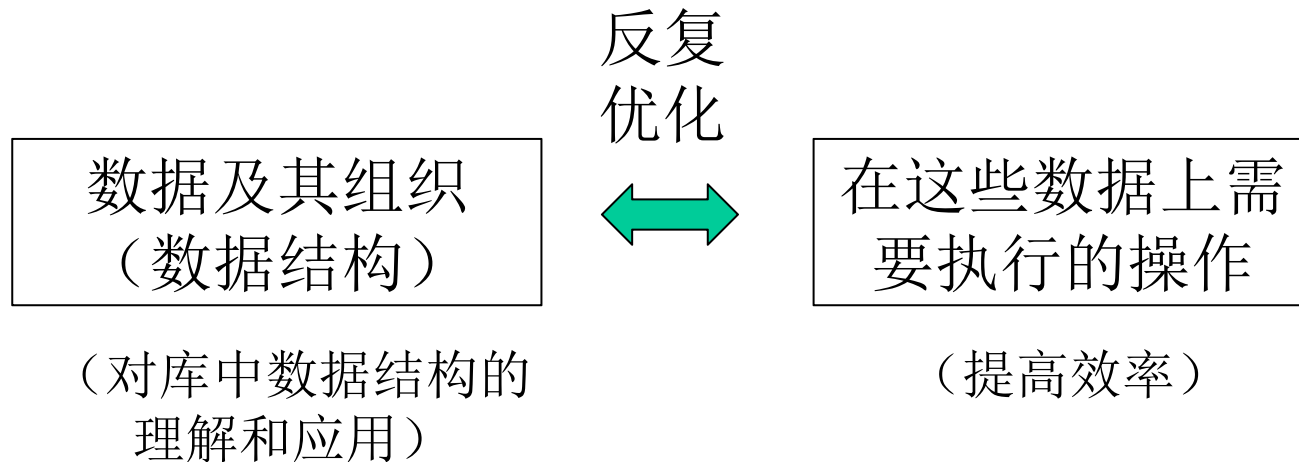


DFA	a	b
$I_0 = \{x, 1, 3, 4\}$	$I_1 = \{1, 3, 2, 4\}$	$I_2 = \{2, 4\}$
$I_1 = \{1, 3, 2, 4\}$	$I_3 = \{3\}$	$I_2 = \{2, 4\}$
$I_2 = \{2, 4\}$	$I_3 = \{3\}$	$I_2 = \{2, 4\}$
$I_3 = \{3\}$	—	$I_2 = \{2, 4\}$



若干主要数据结构（参考）

- 数据结构设计



若干主要数据结构（参考）

- 算法实现
 - 先写伪代码，逐步细化为实际代码
 - 概念清晰、避免一下子陷入局部细节
 - 便于进行算法正确性论证
 - 避免遗漏
 - 便于修改、调整
 - 作为实际代码的注释和实验报告的内容。
 - AI辅助编程场景下
 - 任务分解
 - 使用的技术流程符合编译理论的要求以保持通用性
 - 任务表达
 - 效果校验
 - 网上c代码很多，都可以作为测试对象

若干主要数据结构（参考）

- 设计数据结构存储输入文件第一部分和第三部分的内容
 - 存储Regular Definition
 - 存储第一个%%前的其他声明信息
 - 存储最后一个%%后的函数体
- 存储正规表达式和动作的对应关系

```
typedef struct {  
    string expr; //RE  
    string action; //对应动作（语句）  
} Rule;
```


若干主要数据结构（参考）

- NFA状态
 - 存储NFA状态的编号和出边（ `unordered_multimap` ）。
 - 是否是接受状态。
- NFA
 - 初始状态。
 - 接受状态以及接受状态对应的正规式或动作。
- NFA上的主要操作
 - 合并
 - 用一个新的开始节点通过epsilon边连接各NFA的开始节点
 - 二叉
 - 多叉
 - 遍历

若干主要数据结构（参考）

- DFA状态
 -
 - 所对应的NFA的状态集合（unordered_set ? ）
- DFA
 - 初始状态
- 主要操作：
 - 判断新生成的DFA状态所对应的NFA的状态集是否已经出现过。
 - 通过什么样的数据结构快速获取已经生成的DFA？
 - 如何加快NFA集合的比较速度？
 - 判断接收状态所对应的操作
 - 一个DFA状态对应的NFA状态集中，如果包含多个NFA接收状态，那么应该执行哪个动作？

若干主要数据结构（参考）

- 最小化

- 大步骤通过小步骤实现，每个小步骤实现一次拆分
- 建立数据结构表达等价类，记录状态到等价类的映射关系。
- 从源状态出发，选择一个字符，将一个DFA状态集合（等价类）拆分成多个状态集合
 - 记录状态与变迁后的等价类的映射关系。
- 从目标状态所在的等价类出发，看看各等价类中有哪些状态可以到达该目标等价类。
 - 需要建立方便查找前状态的数据结构。
- 最小化后，实现等价状态的合并。
 - 数据结构应该能表达等价类到该类内状态的映射
 - 建立新的状态（或选一个旧状态作为代表）
 - 合并输入边、合并输出边

若干主要数据结构（参考）

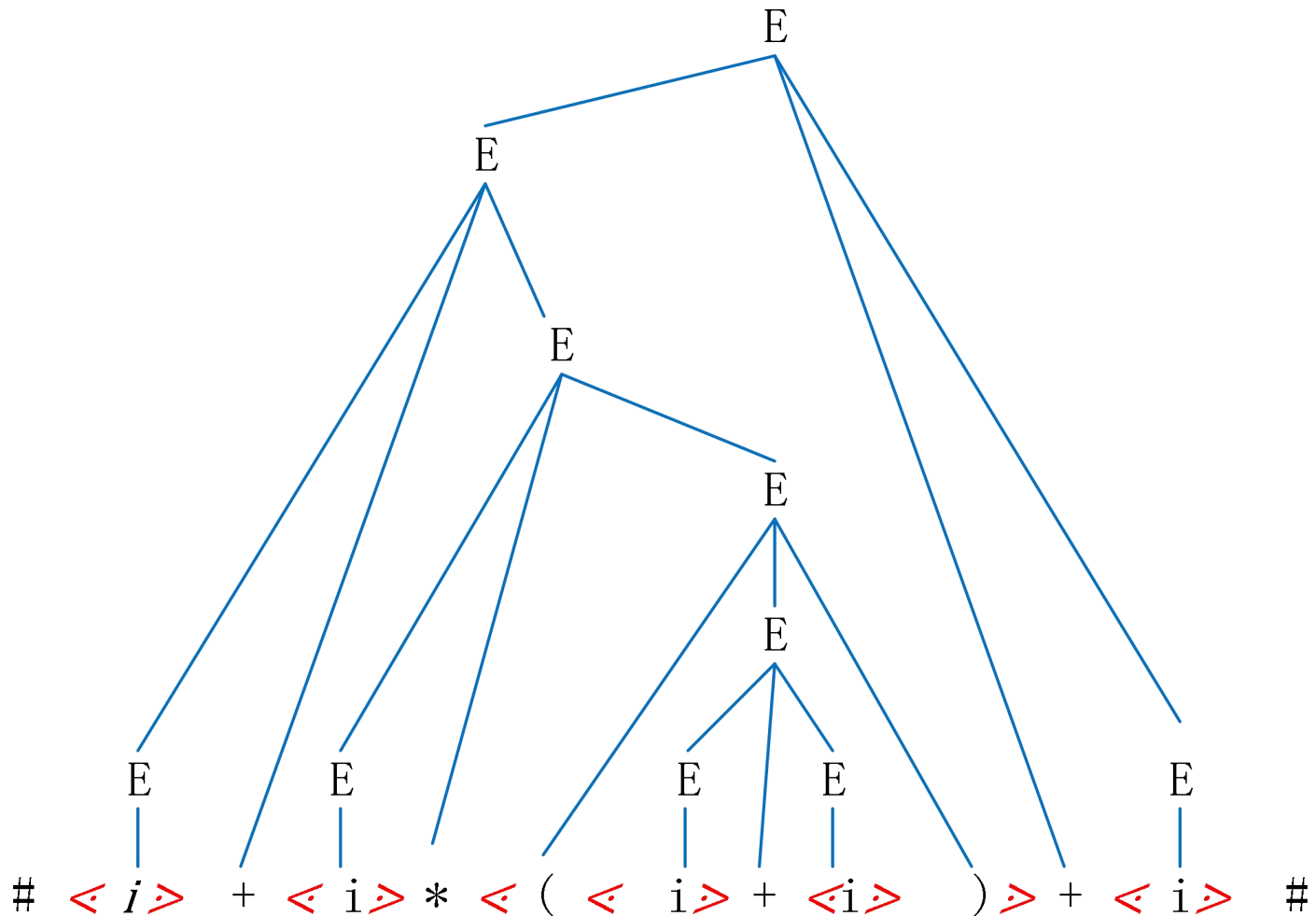
- 词法分析程序生成过程中所涉及的数据结构
 - 识别结果的返回
 - 实现最长匹配（可能需要回退）
 - 记录历史上最近的接受状态
 -

算符优先分析法

- 作用
 - 确定运算符对应的操作数
 - 包括()、[]、{}等定界符的作用范围。

算符优先分析法

- 只比较相邻操作符之间的优先关系



算符优先关系

	+	*	(	)	i	#
+	$\cdot \succ$	$\prec \cdot$	$\prec \cdot$	$\cdot \succ$	$\prec \cdot$	$\cdot \succ$
*	$\cdot \succ$	$\cdot \succ$	$\prec \cdot$	$\cdot \succ$	$\prec \cdot$	$\cdot \succ$
(	$\prec \cdot$	$\prec \cdot$	$\prec \cdot$	$\equiv \cdot$	$\prec \cdot$	
)	$\cdot \succ$	$\cdot \succ$		$\cdot \succ$		$\cdot \succ$
i	$\cdot \succ$	$\cdot \succ$		$\cdot \succ$		$\cdot \succ$
#	$\prec \cdot$	$\prec \cdot$	$\prec \cdot$		$\prec \cdot$	

确定运算符的优先级（可以暂时忽略）

- 设 G 是一个不包含空串产生式的算符文法，并设 $a, b \in V_T$; $P, Q, R \in V_N$ ，定义关系
 - $a \preceq b$ 当且仅当 G 中含有形如 $P \rightarrow \dots ab \dots$ 产生式，或者 $P \rightarrow \dots aQb \dots$ 产生式；
 - 产生式右部不存在相邻的非终结符
 - $a \leq b$ 当且仅当 G 中含有形如 $P \rightarrow \dots aR \dots$ 的产生式，其中 $R \rightarrow b \dots$ ，或 $R \rightarrow Qb \dots$ ；
 - $b \in \text{FirstVT}(R)$ ，后者通过递增法计算（不动点）
 - $a \succ b$ 当且仅当 G 中有形如 $P \rightarrow \dots Rb \dots$ 产生式，其中 $R \rightarrow \dots a$ ，或 $R \rightarrow \dots aQ$ ；
 - $a \in \text{LastVT}(R)$ ，后者通过递增法计算（不动点）
- 若 G 中任何终结符序偶 (a, b) 至多满足上述关系之一，则称 G 为算符优先文法（OPG）

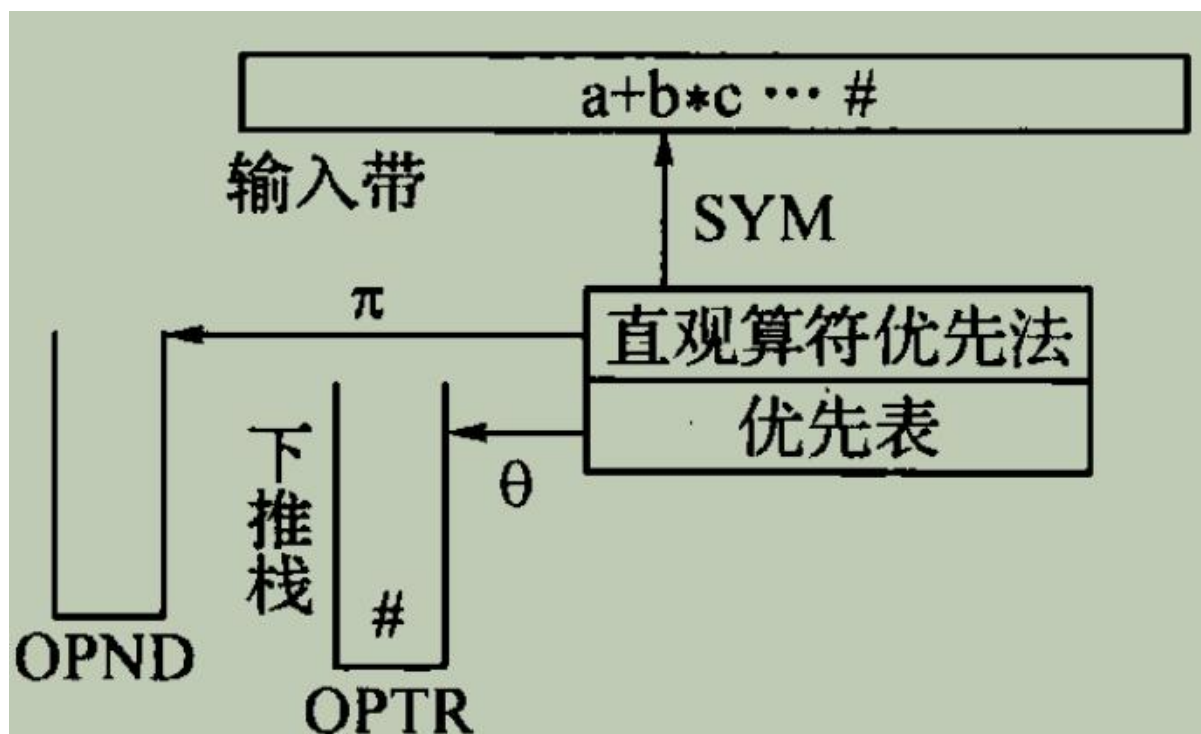


图 5.4 直观算符优先分析法

直观算符优先分析法

- 使用两个工作栈：
 - 一个操作符栈（OPTR）存放运算符和括号，栈顶符号用 θ 表示。
 - 一个操作数栈（OPND）用于存放操作数和运算结果，栈顶符号用 π 表示。

直观算符分析法算法

OPND=“”;

OPTR=“#”;

flag=true;

advance; /*读入一个单词*/

while flag {

 if θ =“#” and SYM=“#” then flag=false /*成功*/

 else if θ =“(“ and SYM=“)” then /*匹配括号对*/

 {OPTR栈顶出栈; advance;}

 else if SYM是操作数 then /*移进操作数*/

 {SYM入OPND栈; advance;}

 else if $\theta \leq$ SYM then /*移进操作符*/

 {SYM入OPTR栈; advance;}

else if $\theta \succ \text{SYM}$ then /*归约*/

{OPND栈顶两个元素 π_1 、 π_2 弹出;

弹出OPTR栈顶元素 θ ;

将 $\pi_1\theta\pi_2$ 规约得到的结果入OPND栈;

}

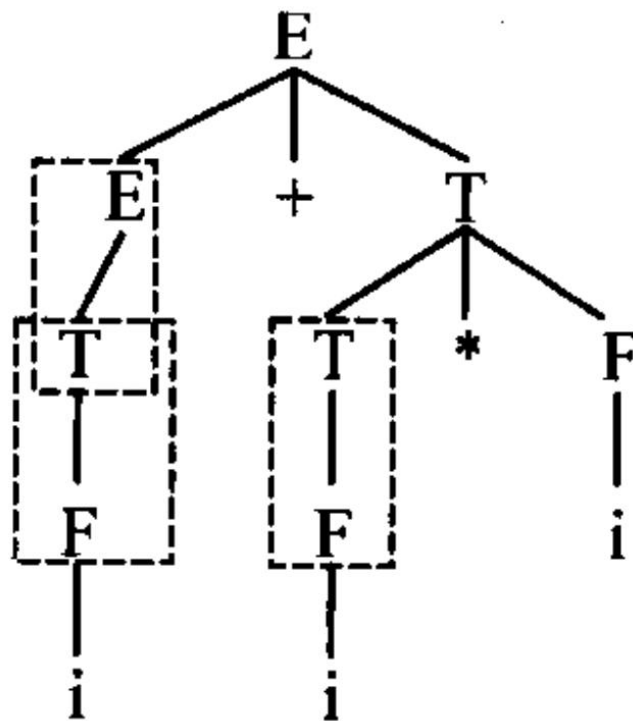
else ERROR;

}}

//在此基础上补充对一元运算符和其他定界符的操作。

算符优先分析法的优、缺点

- 概念简单
- 单目正负号与双目正负号相同，不便处理
 - 事先替换成有区别的符号
- 难以处理不含终结符的产生式。
(含epsilon产生式)



Yacc

- Yacc输入文件的结构

... definition section ...

%%

... rules section ...

%%

... user subroutines section ...

- 前两个部分是必须的，即便其中的一个为空。
- 第三部分以及作为第三部分开始的“%%”可以为空。

Yacc输入文件的定义部分

- 定义部分（definition section）

%{

头文件表

宏定义

数据类型定义（语义值类型定义）

全局变量定义

语法开始符号定义

终结符号定义(tokens)

运算符优先级及结合性定义

%}

符号 (Symbols)

- Yacc语法由符号组成,
 - 由lexer生成的叫做终结符 (*terminal symbols*) 或 词法单元 (*tokens*) 。
 - 全大写字母组成的字符串
 - 非终结符 (*nonterminal symbols* or *non-terminals*)
 - 全小写字母组成的字符串

词法元素（Tokens）

- %token
 - 词法元素（*tokens*）或终结符（*terminal symbols*）是由lexer传递给parser的符号。
 - 每当yacc语法分析程序通过调用**yylex()**得到下一个词法元素。

Tokens

- 有名的词法元素（Symbolic tokens）：
 - 由%**token**定义的符号 (refer to C99.y)
- 以字面值呈现的词法元素（Literal Tokens）：
 - 成对单引号内的字符
 - expr: '(' expr ')';
 - refer to C99.1 and C99.y

词法元素

- 词法元素对应的正整数（Token Numbers）
 - 为每个token对应一个整数，便于处理。
 - 字面值词法元素（literal number）对应的整数通常取对应的ASCII码值。
 - 有名词法元素（Symbolic tokens）对应的整数由yacc输入文件规定
 - %token UP 140 DOWN 150
 - 通常大于可能的单个符号的ASCII码。
 - Lexer需要知道这些token numbers。
 - 放到共享的头文件里

文法开始符号（The Start Symbol）

- 缺省情况下，第一条语法规则左部的符号为文法开始符号。
- 除非在definition section用%start特别声明。
 - Refer to C99.y

运算符及其优先关系、结合律的声明 (Operator Declarations)

- 运算符声明 (Operator declarations) 位于 definitions section 内。
- 可能的声明方式包括 **%left**、**%right** 和 **%nonassoc**。
 - 仅限于词法元素 (操作符)
- 操作符声明按操作符的优先级的升序出现。
 - **%left '+' '-'**
 - **%left '*' '/'**
 - **%nonassoc UMINUS**

%token NAME NUMBER

%left '-' '+'

%left '*' '/'

%nonassoc UMINUS

%%

statement: NAME '=' expression

 | expression { printf("= %d\n", \$1); }

;

expression: expression '+' expression { \$\$ = \$1 + \$3; }

 | expression '-' expression { \$\$ = \$1 - \$3; }

 | expression '*' expression { \$\$ = \$1 * \$3; }

```

|      expression '/' expression
      {      if($3 == 0)
                yyerror("divide by zero");
            else
                $$ = $1 / $3;
      }

```

/ An explicit precedence at the end of a rule.*/*

```

|      '-' expression %prec UMINUS { $$ = -$2; }
|      '(' expression '(' { $$ = $2; }
|      NUMBER { $$ = $1; }
;

```

Yacc解决二义性和冲突的默认方法

- 出现移进/归约冲突时，进行移进
- 出现归约/归约冲突时，按照产生式出现次序，用先出现的产生式归约
- 可以按教材上的方式用优先级定义解决方案

Yacc的规则部分 (Rules Section)

- 产生式
 - Refer to C99.y
- 语义动作

语法分析

- 输入文件的解析
- 根据产生式生成LR(1)状态图
 - 求所有非终结符的First（要递增的滚雪球的计算方式、不应该采用递归调用的方式）
 - 根据需求求某些字符串的First
 - 在一个状态内扩展items的时候要避免循环。
 - 用一个数据结构记录待扩展的items。
 - 看看新扩展的item（含预测符）是否已经有了。
 - 用First计算预测符时，别忘了上下文信息。
 - The kernel items决定一个状态里最终有哪些items，所以，看到kernel items就知道是老状态还是新状态。

语法分析

- 生成LR分析表
 - 处理冲突
- LALR分析
 - 识别同心状态
 - 合并同心状态
 - 生成分析表
- 总控程序（按LR分析表分析输入token串）
- 语法树的输出
 - 可视化，易于阅读，易于判断对错
 - 用markdown实现输出的可视化

调试与测试

- 如何在代码中嵌入调试逻辑？
- 测试对象的生成
 - Csmith
 - a random generator of C programs.
 - It's primary purpose is to find compiler bugs with random programs, using differential testing as the test oracle.
 - <https://github.com/csmith-project/csmith>

所需要的数据结构（供参考）

- 存储根据输入文件获得的产生式数据结构
 - 存储产生式左部到右部的映射
 - 区分语法与嵌入语法的语义部分
 - 拆分由“|”分隔的不同右部
 - 编号
- 存储优先级和结合律的数据结构
- 存储LR(1) item
 - 文法产生式标号
 - 点的位置
 - 预测符集

所需要的数据结构（供参考）

- 存储First的数据结构
 - 非终结符到其First的映射
 - 字符串到其First的映射

所需要的数据结构（供参考）

- 存储LR(1)状态
 - 编号
 - 出边
 - Kernel items及其预测符集
 - Non-kernel items及其预测符集
 - 每当构造一个新状态时，需要判断这个状态是否存在过
 - 比较kernel items及其预测符集即可
 - 排序有益于提高效率。
 - LR(1)-DFA转LALR-DFA时需要判断同心状态
 - 判断kernel items即可
 - 构造Non-kernel items及其预测符集时，需要多次判断某个item是否已经存在，预测符是否已经存在。

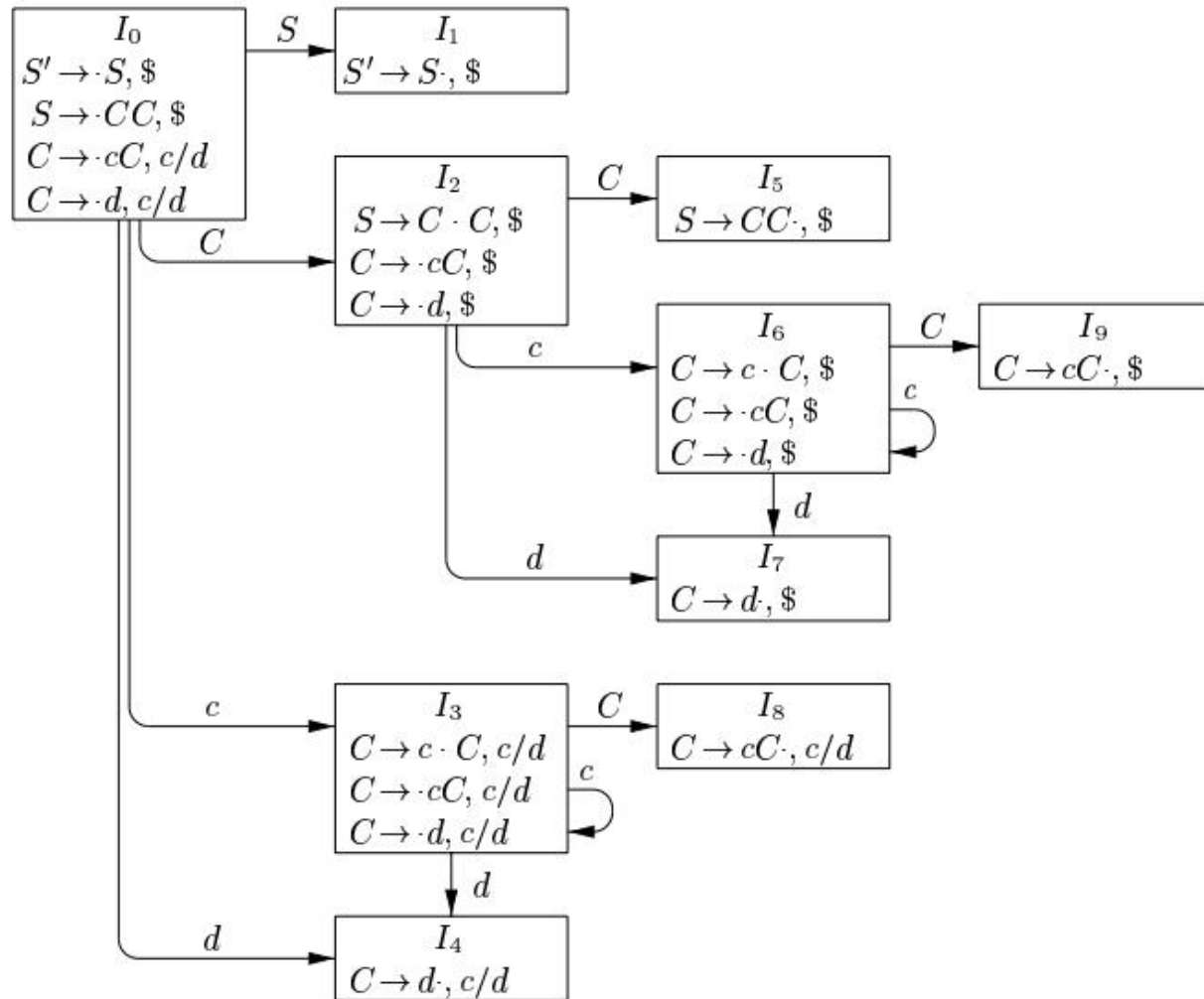
所需要的数据结构（供参考）

- 存储LR(1)DFA
 - 初始状态
 - 状态集
- 存储LR分析表的数据结构
- 存储语法分析结果的数据结构
 - 所使用的产生式序列
 - 语法树
 - 带语义节点的语法树。

4.7.4 LALR (lookahead-LR) Parsing

The Basic Idea

- To reduce the size of LALR parsing table by merging the set of LR(1) items having the same core. (合并同心状态。)



Analysis of Possible Conflicts Caused by the Merging (1)

- shift和GOTO: not influenced by the merging (与预测符无关)
- 只有与reduce有关的行为才可能形成冲突。 -
 - shift-reduce冲突
 - error-reduce冲突
 - reduce-reduce 冲突
 - 对同样的预测符用不同的产生式进行规约。

Analysis of Possible Conflicts Caused by the Merging

- shift-reduce: impossible **Why?**
 - 如果合并后有这样的冲突，那么合并前就已经冲突了，就不是LR(1)文法了。
 - 如果状态 s_1 和 s_2 是同心状态，则二者的移进符集合均为是 $shift(s_{12})$ ，设 s_1 的预测符集是 $lkahd(s_1)$ ， s_2 的预测符集 $lkahd(s_2)$ ，则合并 s_1 和 s_2 之后，有

$$lkahd(s_{12}) = lkahd(s_1) \cup lkahd(s_2),$$

若 $lkahd(s_{12}) \cap shift(s_{12}) \neq \Phi$ ，则

$$(lkahd(s_1) \cup lkahd(s_2)) \cap shift(s_{12}) \neq \Phi,$$

必有

$$lkahd(s_1) \cap shift(s_1) \neq \Phi$$

或

$$lkahd(s_2) \cap shift(s_2) \neq \Phi。$$

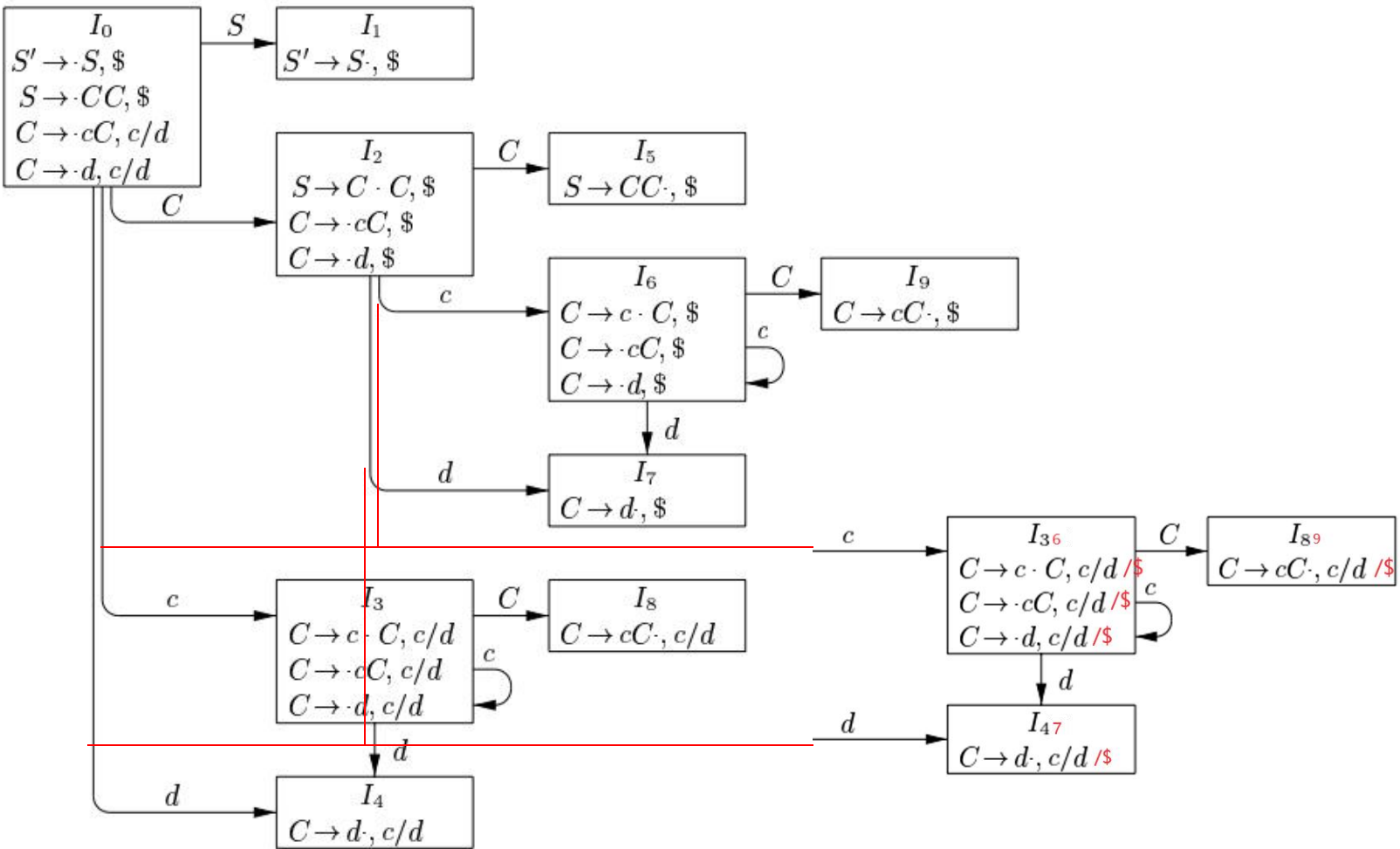
Analysis of Possible Conflicts Caused by the Merging

- error-reduce conflict: does not damage the parsing.
 - merging “ $A \rightarrow B\bullet, a$ ” with “ $A \rightarrow B\bullet, b$ ”, 读头下为a。
 - 晚些报错而已（见下页ppt示例）。

识别: $cd\$$ 合法的输入: $cdcd\$$

合并前: $I_0 \rightarrow I_3 \rightarrow I_4$, 读头下不是c或d, 报错。

合并后: $I_0 \xrightarrow{c} I_{36} \xrightarrow{d} I_{47}$, 按读头下为\$归约 $\rightarrow I_{36} \xrightarrow{C} I_{89}$ 按读头下为\$规约 $\rightarrow I_0 \xrightarrow{C} I_2$, 非规约态, 读头下不是\$, 无法前进, 报错。报错晚一点, 报错点差别不大。



Example of Reduce-Reduce Conflict

- reduce-reduce conflict ?
 - merging “ $A \rightarrow B\bullet, a$ ” with “ $C \rightarrow D\bullet, a$ ”: conflict
 - 不同的状态（彼此同心），可规约短语不同，预测符相同：这也太巧了。
- Example 4.58. Grammar

(0) $S' \rightarrow S$

(1) $S \rightarrow aAd \mid bBf \mid aBe \mid bAe$

(2) $A \rightarrow c$

(3) $B \rightarrow c$

$A \rightarrow c\bullet,$	d
$B \rightarrow c\bullet,$	e

$A \rightarrow c\bullet,$	e
$B \rightarrow c\bullet,$	f

merge



$A \rightarrow c\bullet,$	d/e
$B \rightarrow c\bullet,$	e/f

Construction of LALR Table (Algorithm 4.59)

- Input. An augmented grammar G'
- Output. The LALR parsing table functions ACTION and GOTO for G'
- Method.
 - //总体而言，是在LR(1)自动机里找同心状态，进行合并，然后根据合并后的结果构造分析表。
 - (1) Construct $C = \{I_0, I_1, \dots, I_n\}$, the collection of sets of LR(1) items.
 - (2) For each core present among the set of LR(1) items, find all sets having that core, and replace these sets by their union. (查找同心状态，合并预测符)

- (3) Let $C' = \{J_0, J_1, \dots, J_m\}$ be the resulting sets of LR(1) items. The parsing actions for state i are constructed from J_i in the same manner as in Algorithm 4.56 (参照LR(1)分析表的构建算法, 构建LALR分析表).
 - If there is a parsing action conflict, the algorithm fails to produce a parser, and the grammar is said not to be LALR(1). (有冲突就不是LALR文法)
 - 实践过程中, 没冲突的就合并, 有冲突的就不合并。
- (4) The GOTO table is constructed as follows. If J is the union of one or more sets of LR(1) items, that is, $J = I_1 \cup I_2 \cup \dots \cup I_k$, then the cores of $\text{GOTO}(I_1, X)$, $\text{GOTO}(I_2, X)$, ..., $\text{GOTO}(I_k, X)$ are the same, since $I_1, I_2, \dots, I_k$ all have the same core. (同心状态的规约结果也是同心的。) Let K be the union of all sets of items having the same core as $\text{GOTO}(I_1, X)$. Then $\text{GOTO}(J, X) = K$. (K是 $\text{GOTO}(I_i)$ 合并的结果)。

state	Action			goto	
	c	d	#	S	C
0	S ₃	S ₄		1	2
1			acc		
2	S ₆	S ₇			5
3	S ₃	S ₄			8
4	r ₃	r ₃			
5			r ₁		
6	S ₆	S ₇			9
7			r ₃		
8	r ₂	r ₂			
9			r ₂		

STATE	ACTION			GOTO	
	<i>c</i>	<i>d</i>	\$	<i>S</i>	<i>C</i>
0	s36	s47		1	2
1			acc		
2	s36	s47			5
36	s36	s47			89
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		

Figure 4.43

Parsing string *ccd*\$

也可以在LR(1)自动机里找同心状态，然后对LR(1)分析表进行合并。

- the SLR and LALR tables for a grammar always have the same number of states,
 - typically **several hundred states** for a language like C.
- The canonical LR table would typically have several **thousand states** for the same-size language.

语义分析

词法元素的值

- 由lexer向parser返回词法元素（tokens）。
- 词法元素的值总是存储在变量**yylval**中。
 - 在这个简单的示例中, **yylval**是一个int类型的变量。
[0-9]+ { **yylval** = atoi(yytext); return NUMBER; }
 - 其中: atoi (ascii to integer)把字符串转换成整型数
 - 注意: 上述规则位于lex输入文件中, 可用于计算常量的字面值。

语义动作

- C语言的复合语句（compound statement）
 - date: month '/' day '/' year { printf("date found"); } ;
- 可以引用相应产生式中的符号的语义值。
 - date: month '/' day '/' year
{ printf("date %d-%d-%d found", \$1, \$3, \$5); };

将语义动作嵌入产生式（Embedded Actions）

- `thing: A { printf("seen an A"); } B ;`
等价于

<i>thing:</i>	<i>A fakename B ;</i>
<i>fakename:</i>	<i>/* empty */ { printf("seen an A"); } ;</i>

- 语义动作由一个额外的symbol代表。
 - 无法引用A的语义属性。
- 被嵌入的语义动作可以被想象为对应于规则中的一个dummy symbol，因此其值（在语义动作中被赋给“\$\$”的值）可以被位于产生式末尾的语义动作引用。

<i>thing:</i>	<i>A { \$\$ = 17; } B C { printf("%d", \$2); }</i>
---------------	--

- 匿名语义动作。此处的\$2引用\$\$的值。

继承属性 (Inherited Attributes)

```
declaration:      class type namelist ;

class:            GLOBAL          { $$ = 1; }
                  | LOCAL        { $$ = 2; }
                  ;

type:             REAL            { $$ = 1; }
                  | INTEGER      { $$ = 2; }
                  ;

namelist:         NAME            { mksymbol($0, $-1, $1); }
                  | namelist NAME { mksymbol($0, $-1, $2); }
                  ;

namelist:         NAME            { mksymbol($<tval>0, $<cval>-1, $1); }
                  | namelist NAME { mksymbol($<tval>0, $<cval>-1, $2); }
```

其中： tval和cval为type和class所对应的symbol value的类型。

符号的语义值的数据类型

- Yacc中，词法单元和非终结符均有一个最基本的语义属性。
 - 如果某词法单元是**数值常量**，基本的语义值就是它的表示的数值。
 - 如果是**字符串常量**，基本的语义值可以是指向这个字符串的一个指针
 - 如果是一个符号**SYMBOL**(此处泛指语法分析中的**终结符与非终结符**)，那就可能是一个指向符号表中相应入口的指针。

符号的语义值的数据类型

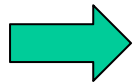
- 语义属性有值，就必须有相应的数据类型。例如，
 - 对数值常量可能是**int**或**double**,
 - 对字符串常量可能是**char ***,
 - 或者指向符号表中相应数据结构的一个指针。
- 这些语义值也需要声明自己的数据类型。

Types of Symbol Value

- Yacc把使用到的每种类型都作为一个C union的成分类型。
- 用**%union**进行声明, 该类型名为**YYSTYPE**.

```
%union {  
    double dval;  
    int vblno;  
}
```

Yacc



```
typedef union {  
    double dval;  
    int vblno; //符号表入口的序号  
} YYSTYPE;  
extern YYSTYPE yylval;  
// extern表示所声明的变量具有全局  
作用域
```

联合类型（Union）

- ```
typedef union {
 int tel_num
 string mail_box
 string address
} contact
```

```
contact a, b, c;
```

```
printf("a.tel_num=%d\n", a.tel_num);
```

```
printf("b.tel_num=%s\n", b.mail_box);
```

```
printf("c.tel_num=%s\n", c.address);
```

# Symbol Value的数据类型的声明

- 对非终结符使用%**type**进行声明。
- 对词法单元使用%**token**声明相应的union field的类型。

# 一个计算器的例子

```
%union {
double dval;
char * sval;
}. //定义了YYSTYPE的类型。所有symbol的symbol value均具有此类型
..
//token REAL的symbol value具有dval分量的类型
%token <dval> REAL
%token <sval> STRING
%type <dval> expr //exp的语义属性的类型
%%
calc: expr { printf(“%g\n”, $1); } //%g用于打印浮点型数据时，会去掉多
//余的零，至多保留六位有效数字。
expr: expr ‘+’ expr { $$ = $1 + $3; }
 | REAL { $$ = $1; } //一个实数常量规约为一个表达式。
 | STRING ‘=’ STRING { $$ = strcmp($1, $3) ? 0.0: 1.0; } //判等。
```

其中，strcmp (\$1, \$3)

- 如果返回值小于 0，则表示 \$1 小于 \$3。
- 如果返回值大于 0，则表示 \$1 大于 \$3。
- 如果返回值等于 0，则表示 \$1 等于\$3。

# C程序三种基本组成成分

- 声明（**declaration**）
  - 告知编译器某个符号（变量、函数、类型）的存在。运行时无需为声明分配内存。
- 定义（**definition**）
  - 给出符号的详细信息，运行时需要为其分配内存。
- 语句（**statements**）

# 声明

- 声明用于声明一个或一组标识符的含义及属性。
- 声明由声明区分符与初值声明符表两部分组成。
- 声明区分符用于指定所声明实体（对象或函数等）的存储持续期（程序运行期、函数调用时）、连接性质（是否可以被其他文件引用）以及类型的部分信息等。
- 初值声明符表用于指定所要声明的实体的名字（标识符）、其它类型信息以及初始化信息等。



# 声明

declaration

: declaration\_specifiers ';' |  
| declaration\_specifiers init\_declarator\_list ';' ;

〈声明〉 ::=

〈声明区分符〉 [ 〈初值声明符表〉 ]

declaration\_specifiers

: storage\_class\_specifier  
| storage\_class\_specifier declaration\_specifiers  
| type\_specifier  
| type\_specifier declaration\_specifiers  
| type\_qualifier  
| type\_qualifier declaration\_specifiers  
| function\_specifier  
| function\_specifier declaration\_specifiers ;

〈声明区分符〉 ::=

〈存储类区分符〉 [ 〈声明区分符〉 ]

| 〈类型区分符〉 [ 〈声明区分符〉 ]

| 〈类型限定符〉 [ 〈声明区分符〉 ]

# 声明

init\_declarator\_list  
: init\_declarator  
| init\_declarator\_list ',' init\_declarator  
;

〈初值声明符表〉 ::=  
    〈初值声明符〉  
    | 〈初值声明符表〉 , 〈初值声明符〉

init\_declarator  
: declarator  
| declarator '=' initializer  
;

〈初值声明符〉 ::=  
    〈声明符〉  
    | 〈声明符〉 = 〈初始化符〉

# 声明

- 例如，在下面的声明中：
- `extern int a[]={1,2,3,4,5,6,7,8,9,0};`
  - `extern int` 是声明区分符（其中 `extern` 为存储类区分符，`int` 为类型区分符）；
  - `a[]={1,2,3,4,5,6,7,8,9,0}`,为初值声明符（初值声明符表中只有这一个初值声明符），其中 `a[]`为声明符，花括号及其括住的部分为初始化符（初始化部分）。

# 声明符

- 声明符用于声明一个实体（对象或函数）。
- 在 c 语中的声明主要由两部分组成:声明区分符与声明符。
- 声明符用于在声明区分符的基础上分层次地构造数组、指针或函数。
  - 例如，设声明区分符所指定的类型为 T,那么可以在此基础上用声明符构造
    - T 的数组、T 的数组的数组、指向 T 的指针、指向 T 的数组的指针、由指向 T 的指针构成的数组、结果类型为 T 的函数、结果类型为指向 T 的指针的函数、指向结果类型为 T 的函数的指针，如此等等。

# 声明符

- 在构造声明符时可以使用一些运算符（也叫构造符），这些运算符在声明中各有着不同的功能，用于构造不同的实体：
  - 数组运算符[]，用于构造数组
  - 函数运算符()，用于构造函数
  - 指针运算符\*，用于构造指针
- 语言规定，编译环境必须至少支持12层的声明符，即一个类型最多可以通过12次复合构造合成。

# 声明符

declarator

: pointer direct\_declarator

| direct\_declarator

;

〈声明符〉 ::=

〈直接声明符〉

| 〈指针〉 〈直接声明符〉

# 声明符

〈直接声明符〉 ::=

〈标识符〉

| ( 〈声明符〉 )

| 〈直接声明符〉 [ ]

| 〈直接声明符〉 [ 〈常量表达式〉 ]

| 〈直接声明符〉 ( 〈参数类型表〉 )

| 〈直接声明符〉 ( )

| 〈直接声明符〉 ( 〈标识符表〉 )

# 声明符

direct\_declarator

: IDENTIFIER

| '(' declarator ')'

| direct\_declarator '[' type\_qualifier\_list assignment\_expression ']'

| direct\_declarator '[' type\_qualifier\_list ']'

| direct\_declarator '[' assignment\_expression ']'

| direct\_declarator '[' STATIC type\_qualifier\_list assignment\_expression ']'

| direct\_declarator '[' type\_qualifier\_list STATIC assignment\_expression ']'

| direct\_declarator '[' type\_qualifier\_list '\*' ']'

| direct\_declarator '[' '\*' ']'

| direct\_declarator '[' ']'

| direct\_declarator '(' parameter\_type\_list ')'

| direct\_declarator '(' identifier\_list ')'

| direct\_declarator '(' ')

;



# 声明符

〈指针〉 ::=

\*

| \* 〈类型限定符表〉

| \* 〈指针〉

| \* 〈类型限定符表〉 〈指针〉

〈类型限定符表〉 ::=

〈类型限定符〉

| 〈类型限定符〉 〈类型限定符表〉

类型限定符: **const** 将标识符限定为不可修改的  
**noalias** 限定标识符不得有别名  
**volatile** 限定标识符为频繁变化的

〈参数类型表〉 ::=

〈参数表〉

| 〈参数表〉, ...

# 声明符

〈参数表〉 ::=  
    〈参数声明〉  
| 〈参数声明〉, 〈参数表〉

〈参数声明〉 ::=  
    〈声明区分符〉  
| 〈声明区分符〉 〈声明符〉  
| 〈声明区分符〉 〈抽象声明符〉

抽象声明符从构造方法与功能上都很类似于声明符，区别仅在于：声明符中要指定所要声明实体的名字（标识符），而抽象声明符中则不指定被声明实体的名字（标识符）

〈标识符表〉 ::=  
    〈标识符〉  
| 〈标识符〉, 〈标识符表〉

# 声明区分符

〈声明区分符〉 ::=

    〈存储类区分符〉 [ 〈声明区分符〉 ]  
| 〈类型区分符〉 [ 〈声明区分符〉 ]  
| 〈类型限定符〉 [ 〈声明区分符〉 ]

`storage_class_specifier`

: `TYPEDEF` //`TYPEDEF`概念上并非存储类区分符，只是借用这个位置

| `EXTERN`

| `STATIC`

| `AUTO`

| `REGISTER`

;

`type_qualifier`

: `CONST`

| `RESTRICT`

| `VOLATILE`

;

# 类型区分符

type\_specifier

: VOID

| CHAR

| SHORT

| INT

| LONG

| FLOAT

| DOUBLE

| SIGNED

| UNSIGNED

| BOOL

| COMPLEX

| IMAGINARY

| struct\_or\_union\_specifier

| enum\_specifier

| TYPE\_NAME ;

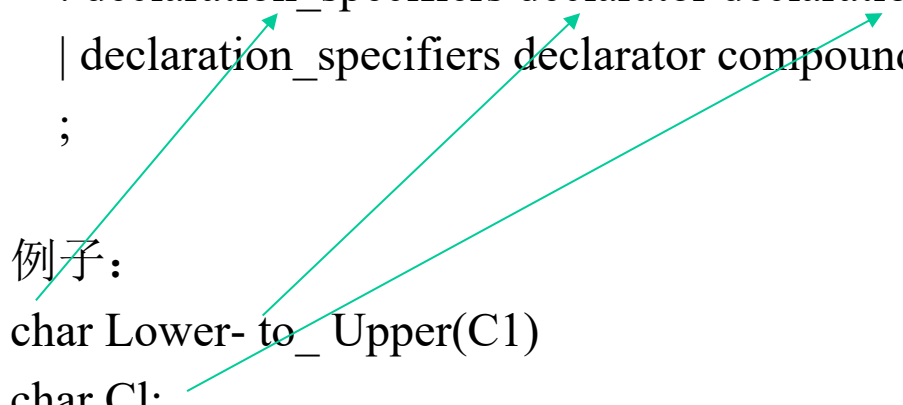
# 定义

function\_definition

: declaration\_specifiers declarator declaration\_list compound\_statement  
| declaration\_specifiers declarator compound\_statement  
;

例子:

```
char Lower_to_Upper(C1)
char C1;
{
 char C2;
 C2=(C1>='a' && C1<='z')?(C1-'a'+'A'): C1;
 return(C2);
}
```



# 定义

`direct_declarator` //函数名和参数

: IDENTIFIER

| '(' declarator ')' //int (\*func)(int); 函数指针

| direct\_declarator '(' parameter\_type\_list ') '

| direct\_declarator '(' ')'

| ...

;

`parameter_type_list`

: parameter\_list

| parameter\_list ',' ELLIPSIS // ELLIPSIS表示省略号，表示可变数目参数

;

# 定义

parameter\_list

: parameter\_declaration

| parameter\_list ',' parameter\_declaration

;

parameter\_declaration

: declaration\_specifiers declarator

| declaration\_specifiers abstract\_declarator

| declaration\_specifiers

;

# 定义

compound\_statement

: '{' '}'

| '{' block\_item\_list '}'

;

block\_item\_list

: block\_item

| block\_item\_list block\_item

;

block\_item

: declaration

| statement

;



# 声明和定义的语义规则

- 声明和定义的语义规则
  - 符号表
  - block的划分
- 声明语句中对类型等信息的处理
  - 参考第六章ppt, 2.7-6.3

## 6.3 Types and Declarations

变量的类型声明语句的翻译，主要工作是

将类型表达式填入符号表

计算各变量在同一个作用域内的相对偏移地址

## 2.7 Symbol Table

符号表的建立和使用

作用域的嵌套结构和符号表的嵌套结构之  
间的对应

符号表的数据结构

进入和退出作用域、声明和使用变量时的  
操作

# Symbol Tables

- Entries in the symbol table contain information about an identifier such as
  - its character string (or lexeme),
  - its type,
  - its position in storage,
  - and any other relevant information.
- typically need to support **multiple declarations of the same identifier** within a program.
  - the scope of a declaration is the portion of a program to which the declaration applies.
  - We shall **implement scopes** by setting up a separate symbol table for each scope.
- **A program block** with declarations will **have its own symbol table** with an entry for each declaration in the block.
  - a class would have its own table, with an entry for each field and method.

## Example 2.14

- a stripped-down language of Java

- {int x; char y; { bool y; x; y; } x; y;}

对x和y的读取操作



- The effect of the declaration (the following is not the symbol table):

- 内层和外层中，读x和y时所看到的x和y的类型为：  
{ { x:int; y:bool; } x:int; y:char; }

## 2.7.1 Symbol Table Per Scope

- Nested Scopes

- *block*  $\rightarrow$  `{decls stmts }` // when *stmts* can generate a block
- The most-closely nested rule
  - Example 2.15 (示例了同一个变量名在不同声明中的出现)
    - The following pseudocode uses subscripts to distinguish among distinct declarations of the same identifier.

1) {    **int**  $x_1$ ; **int**  $y_1$ ;  
 2)    {    **int**  $w_2$ ; **bool**  $y_2$ ; **int**  $z_2$ ;  
 3)         $\dots w_2 \dots$ ;  $\dots x_1 \dots$ ;  $\dots y_2 \dots$ ;  $\dots z_2 \dots$ ;  
 4)    }  
 5)     $\dots \underline{w_0} \dots$ ;  $\dots x_1 \dots$ ;  $\dots y_1 \dots$ ;  
 6) }

注：此处的下标表示变量所对应的声明的层次，即 $y_1$ 、 $y_2$ 是同一变量，都是 $y$ 。

- Chaining of symbol tables results in a tree structure, since more than one block can be nested inside an enclosing block. (从内向外看)

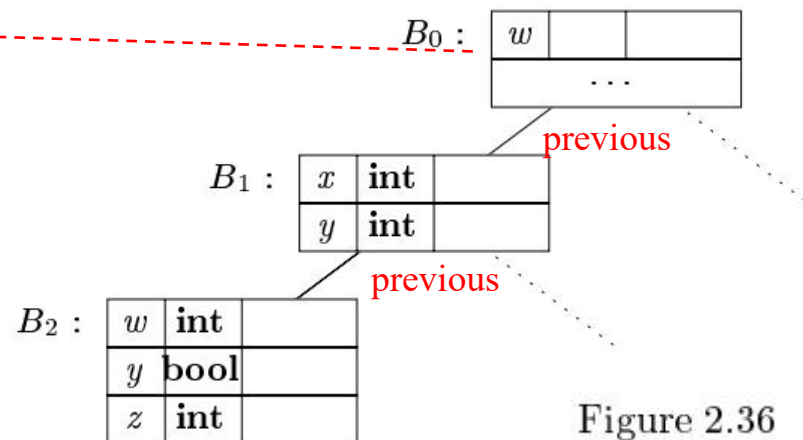


Figure 2.36

# The Java implementation of chained symbol tables

- *Env*(*p*) : 创建符号表
  - Create a new **symbol table**.
    - a hash table named *table*
  - 参数为上层符号表入口
- *Put* a new entry in the current table. 填表(填一行)
  - key-value pairs
  - The key represents a token.
  - The value is an entry of class Symbol.
- *Get* (查表, 返回行入口) an entry for an identifier by searching the chain of tables.
  - Bottom-up

```
1) package symbols; // File Env.java
2) import java.util.*;
3) public class Env { Env代表符号表树上的一个节点。
4) private Hashtable table;
5) protected Env prev;
6) public Env(Env p) {
7) table = new Hashtable(); prev = p;
8) }
9) public void put(String s, Symbol sym) { 此处Symbol代表符号表的一行
10) table.put(s, sym);
11) }
12) public Symbol get(String s) {
13) for(Env e = this; e != null; e = e.prev) {
14) Symbol found = (Symbol)(e.table.get(s));
15) if(found != null) return found;
16) }
17) return null;
18) }
19) }
```

Figure 2.37

## 2.7.2 The Use of Symbol Tables

|                |   |                                                                                  |                                   |
|----------------|---|----------------------------------------------------------------------------------|-----------------------------------|
| <i>program</i> | → | <u>{ <i>top</i> = <b>null</b>; }</u>                                             | <i>null</i> 表示符号表的顶<br>(Virtual)。 |
| <i>block</i>   | → | <i>block</i>                                                                     |                                   |
| <i>block</i>   | → | '{'                                                                              | 进入一个作用域，创建新的一层符号表。                |
|                |   | <u>{ <i>saved</i> = <i>top</i>;<br/><i>top</i> = <b>new Env</b>(<i>top</i>);</u> | <i>top</i> 代表当前的<br>层符号表。         |
|                |   | <i>decls stmts</i> '}'                                                           |                                   |
|                |   | <u>{ <i>top</i> = <i>saved</i>;<br/>print("} "); }</u>                           | 离开一个作用域，返<br>回到上层符号表。             |
| <i>decls</i>   | → | <i>decls decl</i>                                                                |                                   |
|                |   | ε                                                                                |                                   |
| <i>decl</i>    | → | <b>type id ;</b>                                                                 | 创建符号表中的一行。                        |
|                |   | <u>{ <i>s</i> = <b>new Symbol</b>;</u>                                           |                                   |
|                |   | <i>s.type</i> = <b>type.lexeme</b>                                               |                                   |
|                |   | <i>top.put</i> ( <b>id.lexeme</b> , <i>s</i> ); }                                |                                   |



$$\begin{array}{lcl} stmts & \rightarrow & stmts\ stmt \\ & | & \epsilon \end{array}$$

$$\begin{array}{lcl} stmt & \rightarrow & block \\ & | & \underline{factor\ ;} \end{array} \quad \{ \text{print}(";\ "); \}$$

$$\underline{factor} \rightarrow \mathbf{id} \quad \{ \underline{s = top.get(\mathbf{id}.lexeme);}$$

在statement中使用id  
 $\text{print}(\mathbf{id}.lexeme);$   
 $\text{print}(":"); \}$   
 $\text{print}(s.type);$

Figure 2.38: The use of symbol tables for translating a language with blocks

## 6.3.1 Type Expressions

- 抽象性、通用性
- 类型表达式的归纳定义
- 按结构等价和按名等价

## 6.3.1 Type Expressions

- a type expression is either a basic type or is formed by applying an operator called a type constructor to a type expression.
  - （基本类型和复合类型）
  - （类型构造子）

## Example 6.8

- Array type: `int[2][3]`
  - “array of 2 arrays of 3 integers each”
- Type expression: `array(2; array(3; integer))`

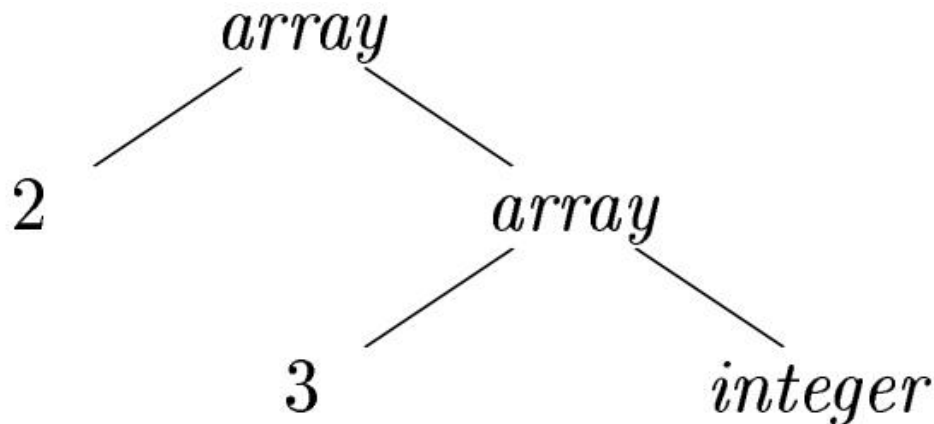


Figure 6.14: Type expression for `int [2] [3]`

# Definition of Type Expressions

## (类型表达式的归纳定义)

- A basic type is a type expression.
  - boolean, char, integer, float, and void
- A type name is a type expression.
  - Class name as type name
    - `public class Node { ... }`  
...  
`public Node n;`
  - Names can be used to define recursive types
    - Pseudo code of a list element
      - `class Cell { int info; Cell next; ... }`

# Definition of Type Expressions

## (类型表达式的归纳定义)

- Applying **the array type constructor** to a number and a type expression.
- Applying **the record type constructor** to the field names and their types.
  - type row = record

address: integer;

lexeme: array[1..15] of char

end;



record((address  $\times$  integer)  $\times$  (lexeme  $\times$  array(1..15, char)))

# Definition of Type Expressions

- Using the type constructor  $\rightarrow$  for function types.
  - $s \rightarrow t$
- If  $s$  and  $t$  are type expressions, then their Cartesian product  $s * t$  is a type expression.
  - to represent a list or tuple of types (e.g., for function parameters)
    - $S_1 * S_2 \rightarrow t$  (\*associates to the left and that it has higher precedence than  $\rightarrow$ )
    - $S_1 \rightarrow S_2 \rightarrow t$  (right associative)
- Type expressions may contain variables whose values are type expressions.

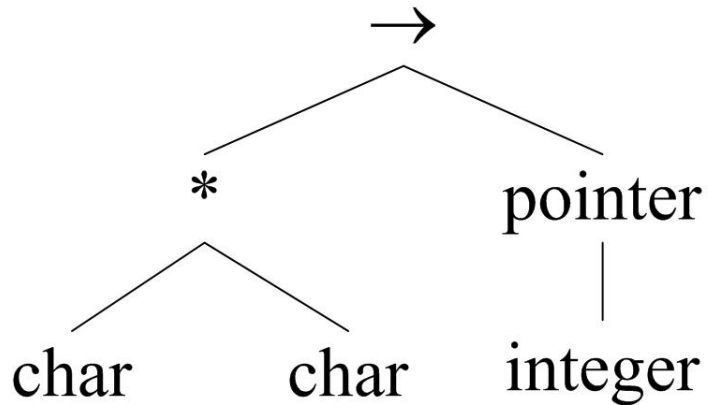
# Example

- $\text{var } p: \uparrow \text{row}$ 
  - $\text{pointer}(\text{row})$
- $\text{mod}(i, j: \text{int}): \text{int}$ 
  - $\text{int} * \text{int} \rightarrow \text{int}$
  - $\text{int} \rightarrow \text{int} \rightarrow \text{int}$
  - $\text{int} \rightarrow (\text{int} \rightarrow \text{int})$

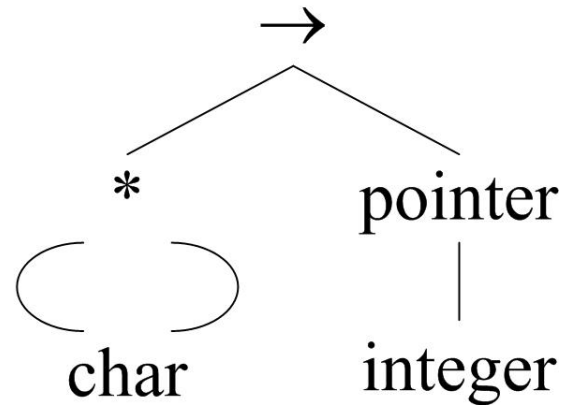


# Represent a Type Expression by a Graph

- function  $f(a, b: \text{char}): \uparrow \text{integer}$ 
  - $\text{char} \times \text{char} \rightarrow \text{pointer}(\text{integer})$



Tree



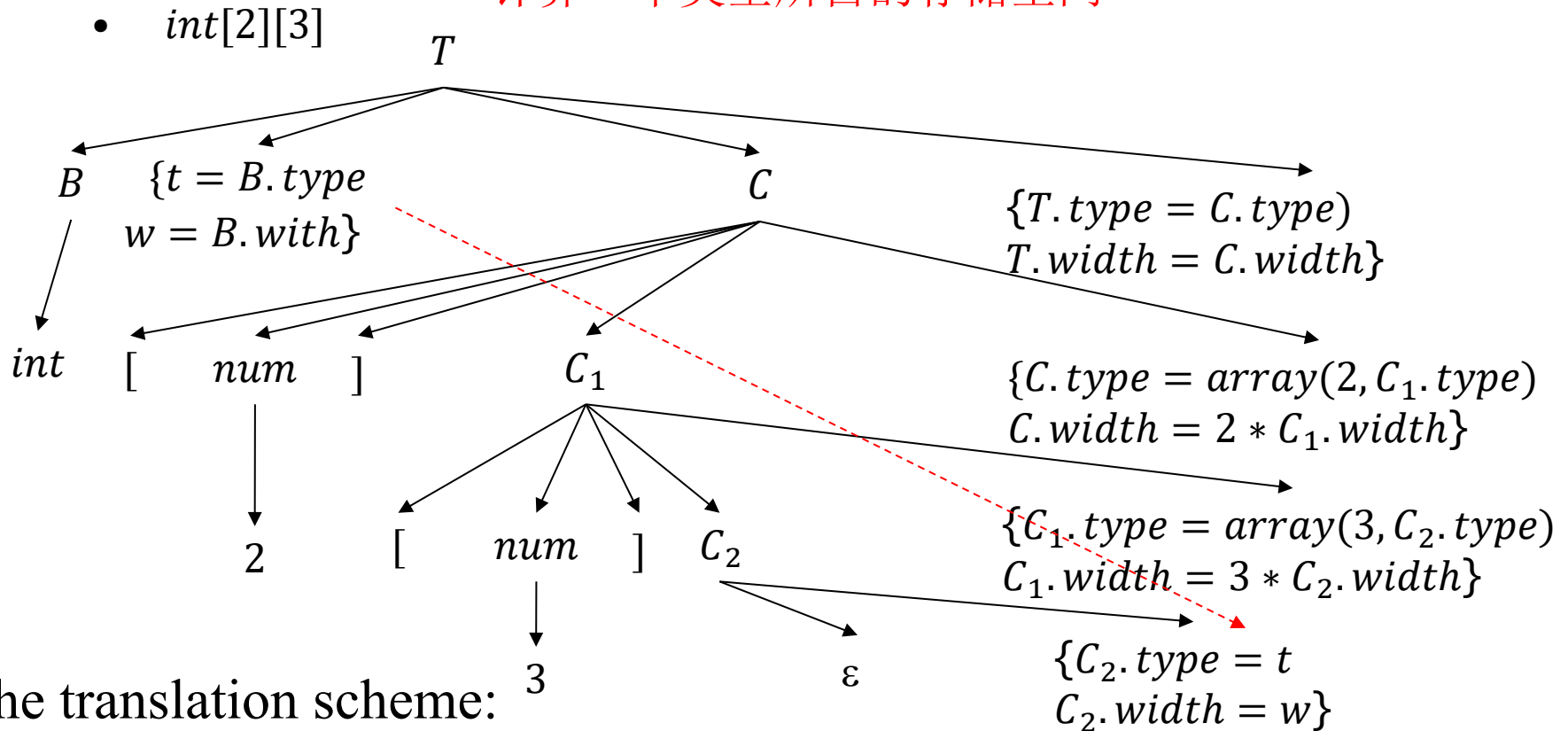
DAG

## 6.3.3 Declarations

- 给定产生式，供下一节使用
- $D \rightarrow T \text{ id} ; D \mid \varepsilon$   
 $T \rightarrow B C \mid \text{record } \{ D \}$   
 $B \rightarrow \text{int} \mid \text{float}$   
 $C \rightarrow \varepsilon \mid [ \text{num} ] C$

## 6.3.4 Storage Layout for Local Names(Example 6.9)

计算一个类型所占的存储空间



$$\begin{array}{l} T \rightarrow B \\ \quad C \end{array} \quad \begin{array}{l} \{ t = B.type; w = B.width; \} \\ \{ T.type = C.type; T.width = C.width; \} \end{array}$$

$$C \rightarrow \epsilon \quad \{ C.type = t; C.width = w; \}$$

$$C \rightarrow [ \mathbf{num} ] C_1 \quad \begin{array}{l} \{ C.type = array(\mathbf{num.value}, C_1.type); \\ C.width = \mathbf{num.value} \times C_1.width; \} \end{array}$$

## 6.3.4 Storage Layout for Local Names(Example 6.9)

注释语法树

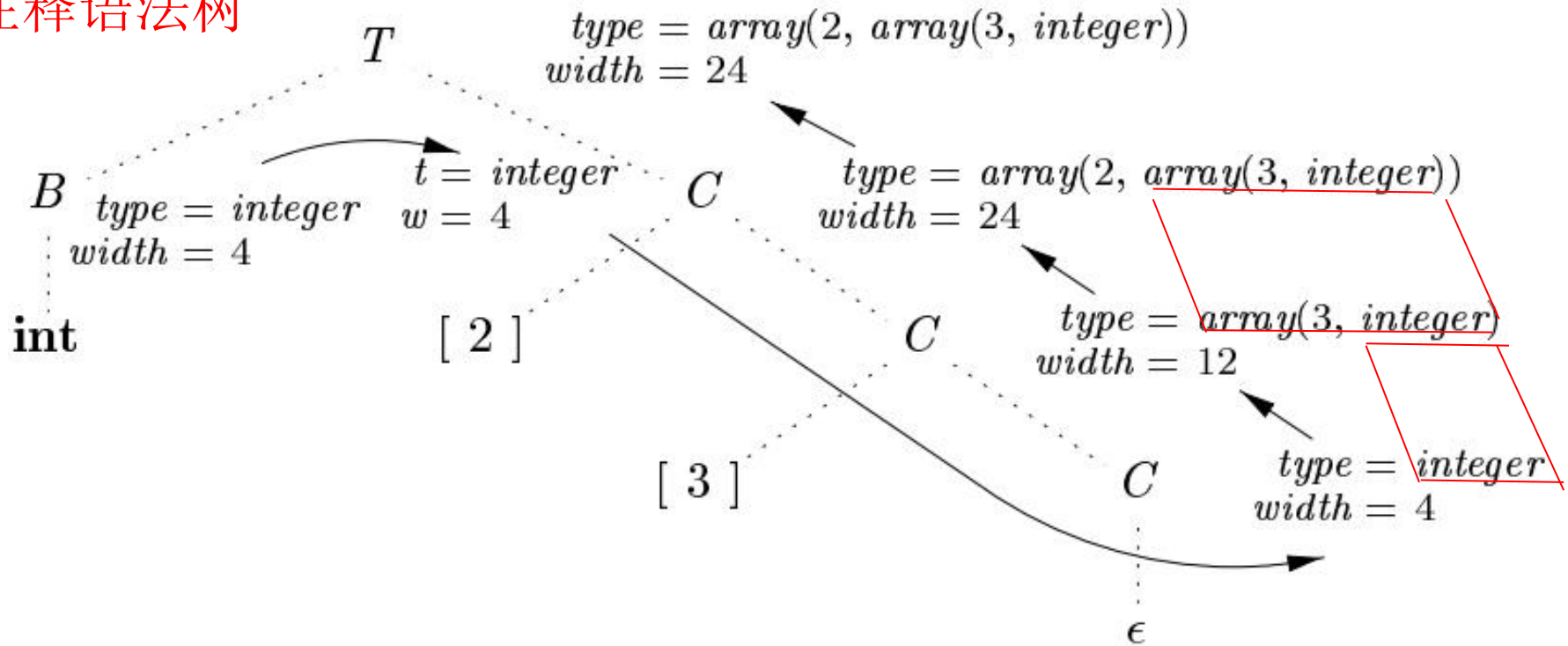


Figure 6.16: Syntax-directed translation of array types

# The Complete Translation Schemes

|                                      |                                                                                                           |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------|
| $T \rightarrow B$                    | $\{ t = B.type; w = B.width; \}$                                                                          |
| $C$                                  | $\{ T.type = C.type; T.width = C.width; \}$                                                               |
| $B \rightarrow \mathbf{int}$         | $\{ B.type = integer; B.width = 4; \}$                                                                    |
| 補充 $B \rightarrow \mathbf{float}$    | $\{ B.type = float; B.width = 8; \}$                                                                      |
| $C \rightarrow \epsilon$             | $\{ C.type = t; C.width = w; \}$                                                                          |
| $C \rightarrow [ \mathbf{num} ] C_1$ | $\{ C.type = array(\mathbf{num.value}, C_1.type);$<br>$C.width = \mathbf{num.value} \times C_1.width; \}$ |

Figure 6.15: Computing types and their widths

For the explanation, refer to the next page.

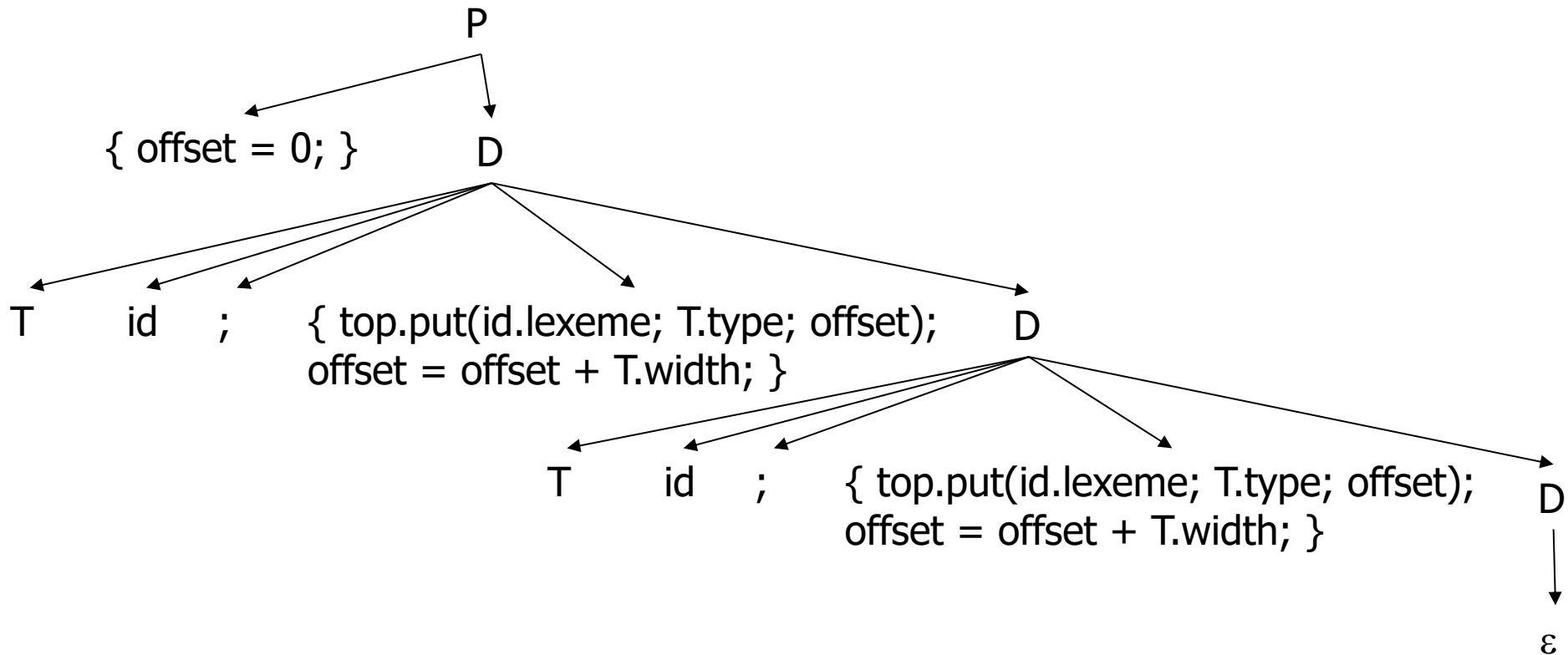
## 6.3.5 Sequences of Declarations

- 变量在作用域内的相对地址偏移量按声明的顺序累加
- Symbol table
  - Refer to slides *Symbol Table*
  - *top* denotes the current symbol table.

# Offset (计算每个局部变量在本函数内的偏移地址)

$$P \rightarrow D \quad \{ \text{offset} = 0; \}$$

$$D \rightarrow T \text{ id } ; \quad \{ \text{top.put}(\text{id.lexeme}, T.\text{type}, \text{offset}); \\ \text{offset} = \text{offset} + T.\text{width}; \}$$

$$D \rightarrow \epsilon$$


## 6.3.6 Fields in Records and Classes

- Record或Class本身也形成一个作用域。
- $T \rightarrow \text{record } \{ D \}$ 
  - offset of the fields
    - The offset or relative address for a field name is relative to the data area for that record.



# Fields in Records and Classes(自习)

$T \rightarrow \text{record '}' \{ \text{Env.push}(top); top = \text{new Env}();$   
 $\text{Stack.push}(offset); offset = 0; \}$   
 $D '}' \{ T.type = \text{record}(top); T.width = \text{offset};$   
 $\text{top} \text{ 代表的类型表达式}$   
 $\text{top} = \text{Env.pop}(); \text{offset} = \text{Stack.pop}(); \}$   
 $\text{record 内累积的 offset}$   
 $\text{恢复在上一级中的 offset}$

- **pseudocode** for a specific implementation using stack (**not mandatory**).
- class *Env* implement symbol tables.
- *Env.push(top)* pushes the current symbol table denoted by *top* onto a stack.
- Variable *top* is then set to a new symbol table.
- *offset* is pushed onto a stack called *Stack*.

# 语句 (statement)

- 参考第6章ppt
  - 三地址语句和四元式
    - 6.2节
- 函数调用
- Procedure calls
  - $p(x_1; x_2; \dots; x_n)$  (高级语言中的函数调用)
    - $param\ x_1$   
 $param\ x_2$   
 $\dots$   
 $param\ x_n$   
 $call\ p, n$
  - $y = call\ p, n$  (函数调用的结果赋值给变量)
- return
  - $return\ y$

# 可能需要的数据结构（供参考）

- 符号表（可以根据语言的不同有所区别）
  - 指向父符号表的指针
  - 所处的作用域（全局、某函数、块语句、结构）及相关信息，以函数为例：
    - 函数名称
    - 参数数量、类型、名称、所占空间大小、起始地址（相对于sp）
    - 返回值类型、所占空间
    - 对应的源代码的行列范围
  - 变量表
    - 局部变量起始地址(相对于sp)
    - 局部变量类型表达式、名称、所占地址
    - 对应的源代码的行列范围
  - 中间代码的入口序号和出口序号
  - 符号表的可视化、填表、查表的函数或方法

# 可能需要的数据结构（供参考）

- 扩展语法树存储语义值
  - 根据语法元素在产生式中的相对位置查找语义值
- 存储三地址语句或四元式的数据结构
  - 申请临时变量的方法
  - 并链的方法
  - 回填的方法

# LLVM IR

- LLVM是一个正在发展中的前沿编译器技术框架，它易于扩展并设计成多个库，可以为编译器入门者提供流畅的体验，并能使编译器开发所涉及的学习过程变得非常顺畅。
- 用于编译器相关研究。
- LLVM IR 被设计成尽可能地与编译目标无关，但它仍然对编译目标有一定的依赖性。
  - LLVM用C++开发。
  - 造成该依赖性的一个普遍承认的原因是 C/C++ 语言固有的目标依赖性质。
    - 所依赖的库头文件和 C 的类型都已经是依赖目标的。

# LLVM IR

- LLVM IR 可以用两种格式存储在磁盘上：位码和汇编文本。
- 例：
  - `int sum(int a, int b) { return a+b; }`
  - 其汇编码文件 `sum.ll` 内容如下页所示

target datalayout: 目标三元组 (target trippie) 的字节顺序和类型大小等信息，以供优化。可以忽略。

target triple: 目标机器的硬件系统。可以忽略。

```
target datalayout = "e-p:64:64:64-i1:8:8-i8:8:8-i16:16:16-
 i32:32:32-i64:64:64-f32:32:32-f64:64:64-v64:64:64-v128:128:128-
 a0:0:64-s0:64:64-f80:128:128-n8:16:32:64-S128"
target triple = "x86_64-apple-macosx10.7.0"
```

```
define i32 @sum(i32 %a, i32 %b) #0 {
entry:
```

```
 %a.addr = alloca i32, align 4
```

为形参分配存储空间。

```
 %b.addr = alloca i32, align 4
```

```
 store i32 %a, i32* %a.addr, align 4
```

实现参数传递

```
 store i32 %b, i32* %b.addr, align 4
```

```
 %0 = load i32* %a.addr, align 4
```

完成运算

```
 %1 = load i32* %b.addr, align 4
```

```
 %add = add nsw i32 %0, %1
```

```
 ret i32 %add
```

nsw是not signed wrap的缩写，表示保证不会发生有符号数运算的溢出。

```
attributes #0 = { nounwind ssp uwtable ... }
```

# LLVM IR

- 整个 LLVM 文件的内容（无论是汇编码还是位码）被视为定义一个 LLVM 模块。
- 模块是 LLVM IR 顶层数据结构。
  - 每个模块包含一系列函数，
  - 每个函数由包含一系列指令的一系列基本块组成。
  - 模块还包含用于支持该模型的外围实体，如全局变量、目标数据布局、外部函数原型以及数据结构声明。



# LLVM IR

- LLVM 局部变量与汇编语言中的寄存器类似，可以用任何以 % 符号开头的名称命名。
- 因此，

`%add = add nsw i32 %0, %1`

这一指令将执行两个局部变量 %0 和 %1 的加法，并将结果置于新的局部变量 %add 中。

# LLVM IR

- LLVM IR使用单复制（Static Single-Assignment Form, SSA）形式。

```
p = a + b
q = p - c
p = q * d
p = e - p
q = p + q
```

```
p1 = a + b
q1 = p1 - c
p2 = q1 * d
p3 = e - p2
q2 = p3 + q1
```

(a) Three-address code.

(b) Static single-assignment form.

# LLVM IR

- 导致“使用定义链”（use.def链，即可以达到使用处的所有定义 / 赋值语句的集合）的生成变得非常简单。
- 代码被组织成三地址指令。
- 有无穷多的寄存器。请注意，LLVM 局部变量可以使用以 % 符号开始的任意名称, 例如%0、%1等从零开始的数字。

# LLVM IR

- 函数声明严格遵循相应的 C 语法：

```
define i32 @sum(i32 %a i32 %b) #0 {
```

- 此函数返回 i32 类型的值，并具有两个 i32 参数： %a 和 %b。
- 本地标识符始终需要 % 前缀，而全局标识符使用 @。
- 函数声明中的 #0 记号映射到一组函数属性，
- 这组属性被定义在文件末尾（暂时忽略）：

```
attributes #0 = { nounwind ssp uwtable "less-precise-fpmad"="false" "no-frame-pointer-elim"="true" "no-frame-pointer-elim-non-leaf"="true" "no-infs-fp-math"="false" "no-nans-fp-math"="false" "unsafe-fp-math"="false" "use-soft-float"="false" }
```

# LLVM IR

- 函数的主体部分明确划分为多个基本块 ( basic block, BB), 而每个新的基本块开始处都有一个标签。
- 标签与基本块的关系就如同变量标识符与指令。
- 代码跳转到对应于基本块的标签。
- 每个基本块都需要以一个结束符指令结尾, 结束符一般为跳转到另一个基本块或从该函数返回。
- 第一个基本块称为入口基本块, 它在 LLVM 函数中的地位很特殊, 不能作为任何分支指令的目标。

# LLVM IR

- sum.ll 只有一个基本块

entry:

`%a.addr = alloca i32, align 4`

`%b.addr = alloca i32, align 4`

`store i32 %a, i32* %a.addr, align 4`

`store i32 %b, i32* %b.addr, align 4`

`%0 = load i32* %a.addr, align 4`

`%1 = load i32* %b.addr, align 4`

`%add = add nsw i32 %0, %1`

`ret i32 %add`

}

为形参分配存储空间。

实现参数传递

完成运算

# LLVM IR

- 第一条指令 `%a.addr=alloca i32, align 4` 分配一个按 4 字节对齐的 4 字节堆栈元素。
  - 指向该堆栈元素的指针被存储在本地变量 `%a.addr` 中。
- `alloca` 指令通常用于表示本地（自动）变量。
- `%a`和 `%b`参数通过 `store` 指令存储在堆栈地址 `%a.addr` 和 `%b.addr` 中。
- 这些值通过 `load` 指令从相同的内存地址加载回来。

# 编写自定义的 LLVM IR 生成器

- 使用 LLVM IR 生成器 API 以编程方式为 `sum.ll` 构建 IR。
- 所需头文件
  - 见《LLVM编译器实战教程》5.4节
- 在这个例子中，我们也从 LLVM 命名空间中导入符号：`using namespace llvm;`
- 之后我们分步骤编写代码，生成 `sum.ll` 的不同部分。



# 编写自定义的 LLVM IR 生成器

1. 要编写的第一段代码是定义一个名为 `makeLLVMModule` 的新辅助函数，它返回指向 `Module` 实例的指针，该 `Module` 实例是包含所有其他 IR 对象的顶层实体：

```
Module *makeLLVMModule() {
 Module *mod = new Module("sum.ll", getGlobalContext());
 mod->setDataLayout("e-p:64:64:64-i1:8:8-i8:8:8-i16:16:16-
 i32:32:32-i64:64:64-f32:32:32-f64:64:64-v64:64:64-
 v128:128:128-a0:0:64-s0:64:64-f80:128:128-
 n8:16:32:64-S128");
 mod->setTargetTriple("x86_64-apple-macosx10.7.0");
}
```

- 通过 `getGlobalContext()` 获取当前 LLVM 上下文，并定义模块的名称。
- 其它两个函数可以舍弃。

# 编写自定义的 LLVM IR 生成器

## 2. 为了声明求和函数，首先定义如下函数签名（signature）：

```
SmallVector<Type*, 2> FuncTyArgs; 声明一个存放函数参数的向量。
FuncTyArgs.push_back(IntegerType::get(mod->getContext(),
 32)); 指定第1个参数的类型为32位的整数。
FuncTyArgs.push_back(IntegerType::get(mod->getContext(),
 32)); 指定第2个参数的类型为32位的整数。
FunctionType *FuncTy = FunctionType::get(
 /*Result=*/ IntegerType::get(mod->getContext(), 32),
 /*Params=*/ FuncTyArgs, /*isVarArg=*/ false);
指定函数的返回类型为32位的整数，指定函数的参数类型。
```

# 编写自定义的 LLVM IR 生成器

3. 接下来我们使用 `Function::Create()` 静态方法创建一个函数。

```
Function *funcSum = Function::Create(
 /*Type=*/ FuncTy, 指定函数的signature。
 /*Linkage=*/ GlobalValue::ExternalLinkage,
 /*Name=*/ "sum", mod); 该函数可以被其他模块 (编译单元) 引用。
funcSum->setCallingConv(CallingConv::C);
```

LLVM缺省的调用方式（calling convention），和C语言一致。

# 编写自定义的 LLVM IR 生成器

4. 接下来，我们需要存储参数的 Value 指针，以便以后使用它们。

函数参数迭代器

```
Function::arg_iterator args = funcSum->arg_begin();
Value *int32_a = args++; 为对函数参数的引用提供别名 (alias)
int32_a->setName("a"); 为函数参数命名
Value *int32_b = args++;
int32_b->setName("b");
```

# 编写自定义的 LLVM IR 生成器

5. 在开始函数体之前，我们用entry 标签 (或值名称) 创建第一个基本块，并在labelEntry 中存储它的指针。

```
BasicBlock *labelEntry = BasicBlock::Create(mod->getContext(), "entry", funcSum, 0);
```

该基本块被安插在函数的末尾(当参数 InsertBefore 为 0 时)。

# 编写自定义的 LLVM IR 生成器

## 6. 向基本块添加两条 `alloca` 指令，以创建一个 4 字节对齐的 32 位堆栈元素。

- 默认情况下，新的指令被插入基本块的末尾。

```
// Block entry (label_entry)
AllocaInst *ptrA = new AllocaInst(IntegerType::get(mod-
 >getContext(), 32), "a.addr", labelEntry);
ptrA->setAlignment(4);
AllocaInst *ptrB = new AllocaInst(IntegerType::get(mod-
 >getContext(), 32), "b.addr", labelEntry);
ptrB->setAlignment(4);
```

- 生成结果:

```
entry:
 %a.addr = alloca i32, align 4
 %b.addr = alloca i32, align 4
```

# 编写自定义的 LLVM IR 生成器

7. 我们使用 `alloca` 指令返回的指针 `ptrA` 和 `ptrB` 将函数参数 `int32_a` 和 `int32_b` 存储到堆栈位置中。

```
StoreInst *st0 = new StoreInst(int32_a, ptrA, false,
 labelEntry);
st0->setAlignment(4);
StoreInst *st1 = new StoreInst(int32_b, ptrB, false,
 labelEntry);
st1->setAlignment(4);
```

- 第三个 `StoreInst` 参数指定这是否是一个非静态存储区，在此示例中为 `false`。

# 编写自定义的 LLVM IR 生成器

8.

```
LoadInst *ld0 = new LoadInst(ptrA, "", false,
 labelEntry); 创建静态加载指令，从堆栈中将参数取出。
ld0->setAlignment(4);
LoadInst *ld1 = new LoadInst(ptrB, "", false,
 labelEntry);
ld1->setAlignment(4);
BinaryOperator *addRes =
 BinaryOperator::Create(Instruction::Add, ld0, ld1,
 "add", labelEntry); 创建加法指令，将取出的参数相加。
ReturnInst::Create(mod->getContext(), addRes,
 labelEntry); 创建返回指令，将相加的结果返回。

return mod;
}
```

接下来，makeLLVMModule 函数返回被创建的模块mod和我们刚创建的 sum 函数。



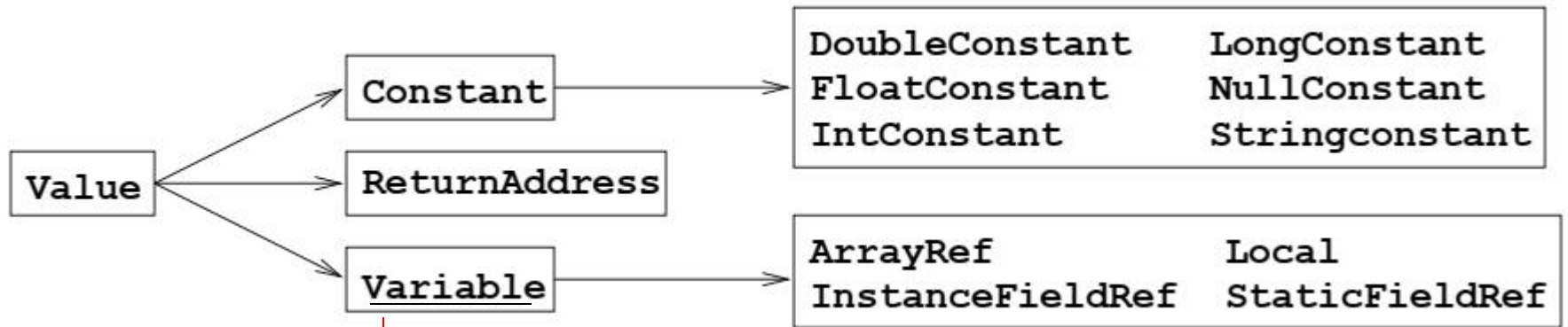
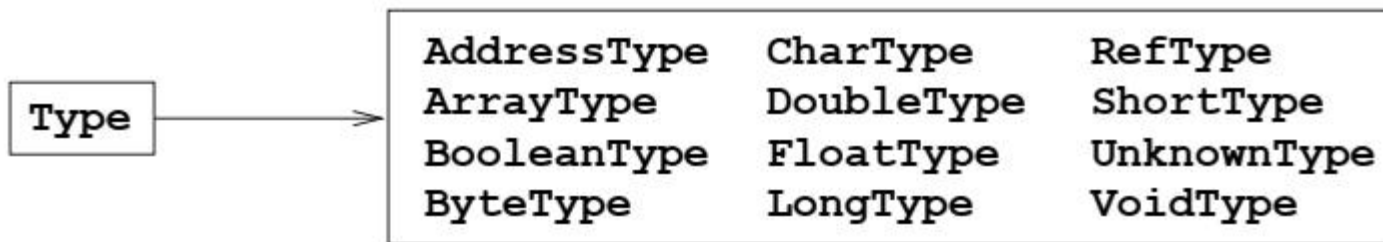
# 编写自定义的 LLVM IR 生成器

9. 为了使 IR 生成器程序成为一个独立的工具，它还需要一个 `main()` 函数。

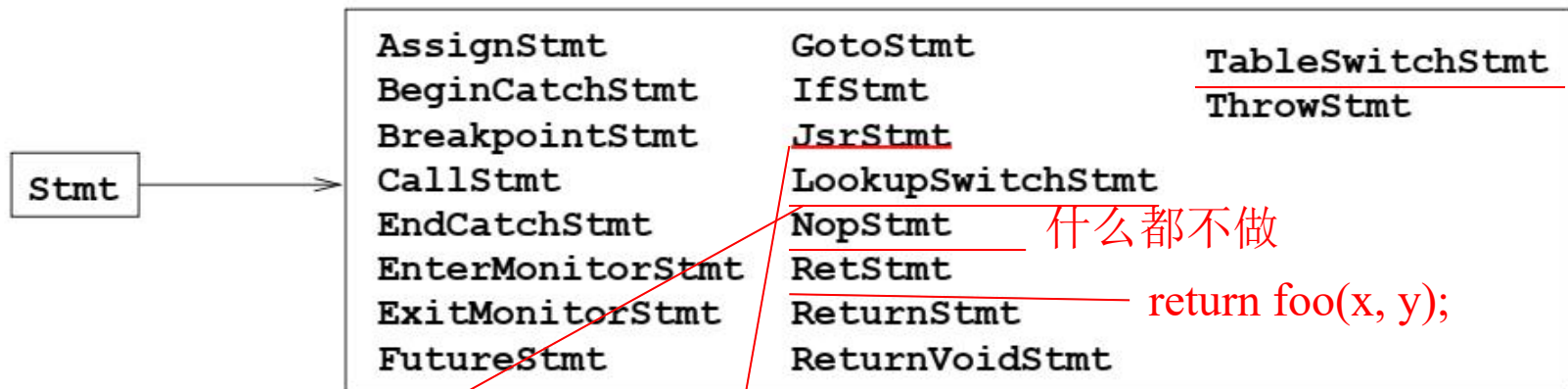
```
int main() {
 Module *Mod = makeLLVMModule(); 这个函数在前面定义过的，
这里是调用。
 verifyModule(*Mod, PrintMessageAction); 验证该模块
 std::string ErrorInfo; 错误信息
 OwningPtr<tool_output_file> Out(new tool_output_file(
 "/sum.bc", ErrorInfo, 创建输出流。
 sys::fs::F_None));
 if (!ErrorInfo.empty()) {
 errs() << ErrorInfo << '\n';
 return -1;
 } 将bit code写入输出文件。
 WriteBitcodeToFile(Mod, Out->os()); 返回Out中包含的输出流。
 Out->keep(); // Declare success
 return 0;
}
```

## 中间表示: Jimple

- Jimple是一种有类型的、基于三地址语句的中间表示
  - 相应接口在包soot.jimple、soot.jimple.internal中
- Jimple可以转换为Java字节码执行。
  - soot.jimple.JasminClass



Ivalues, locations that can contain values.



JsrStmt代表一个jsr指令，以跳转到一个实现  
exception handlers中的finally clause的子程序。

switch语句的编译结果:

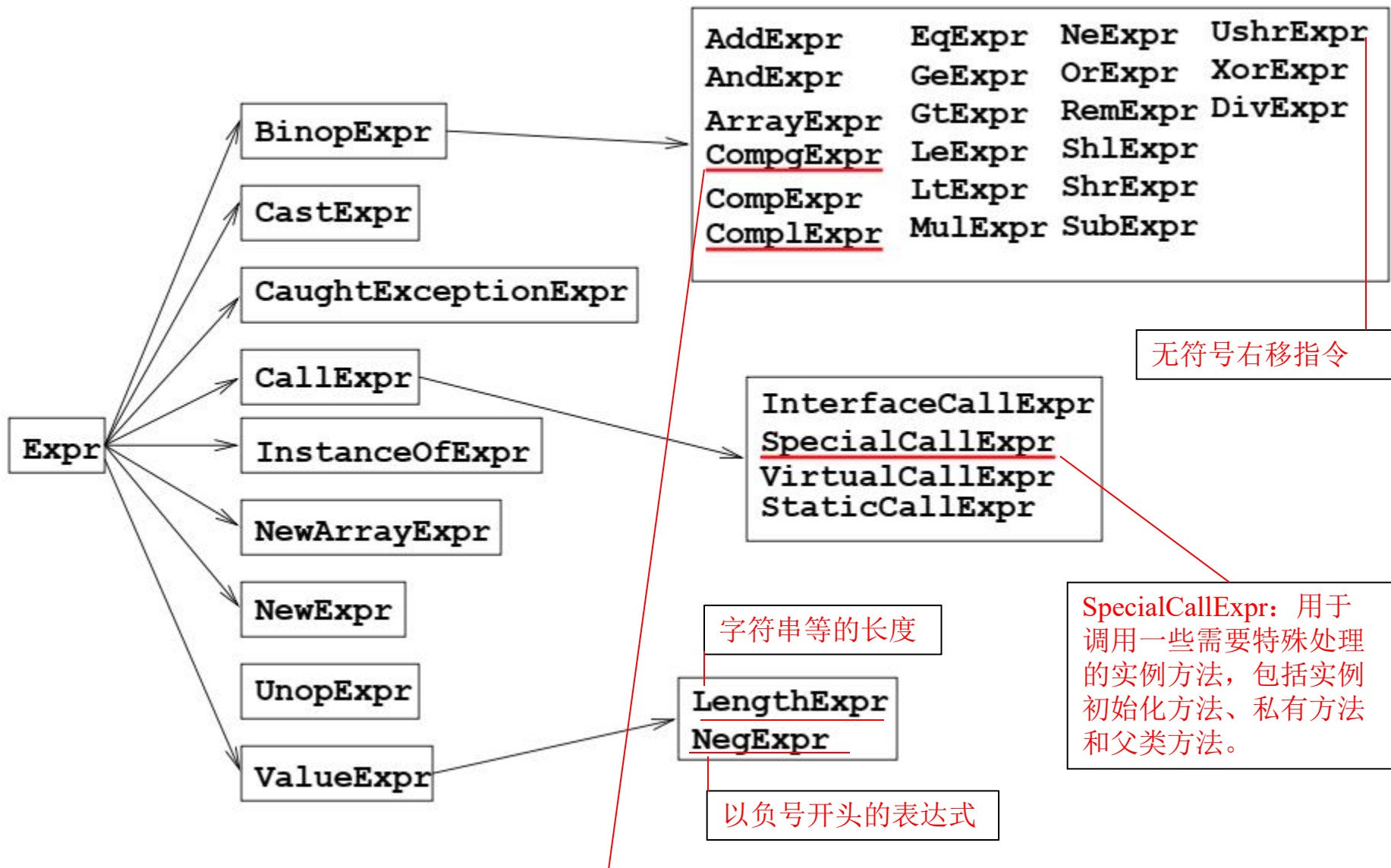
TableSwitchStmt

LookupSwitchStmt

```
switch (inputValue) {
 case 1: // ...
 case 2: // ...
 case 3: // ...
 default: // ...
}
```

```
switch (inputValue) {
 case 1: // ...
 case 10: // ...
 case 100: // ...
 case 1000: // ...
 default: // ...
}
```

在执行tableswitch时，堆栈顶部的int值直接用作表中的索引，以获取跳转目标并立即执行跳转。  
执行lookupswitch时，将堆栈顶部的int值与表中的键进行比较，直到找到匹配项，然后使用该键旁边的跳转目标执行跳转。



**cmpf**-float vAA,vBB,vCC

比较两个单精度的浮点数,如果vBB寄存器中的值大于vCC寄存器的值,则返回-1到vAA中,相等则返回0,小于返回1

cmpg-float vAA,vBB,vCC

比较两个单精度的浮点数,如果vBB寄存器中的值大于vCC的值,则返回1,相等返回0,小于返回-1

**cmpd**-double vAA,vBB,vCC

比较两个双精度浮点数,如果vBB寄存器中的值大于vCC的值,则返回-1,相等返回0,小于则返回1

## Java Code

```
public static void main(String[] argv)
throws Exception
{
 int x = 2, y = 6;

 System.out.println("Hi!");
 System.out.println(x * y + y);
 try
 {
 int z = y * x;
 }
 catch (Exception e)
 {
 throw e;
 }
}
```

无异常正常返回

表达异常的捕捉范围和响应者

局部变量，对应  
x, y和z

```
public static void main(java.lang.String[]) throws java.lang.Exception
{
 java.lang.String[] r0;
 int i0, i1, i2, $i3, $i4;
 java.io.PrintStream $r1, $r2;
 java.lang.Exception $r3, $r4;

 r0 := @parameter0;
 i0 = 2;
 i1 = 6;
 $r1 = java.lang.System.out;
 $r1.println(`Hi!`);
 $r2 = java.lang.System.out;
 $i3 = i0 * i1;
 $i4 = $i3 + i1;
 $r2.println($i4);

 label0:
 i2 = i1 * i0;

 label1:
 goto label3;

 label2:
 $r3 := @caughtexception;
 r4 = $r3;
 throw r4;

 label3:
 return;

 catch java.lang.Exception from label0 to label1 with label2;
}
```

存放输入变量

\$开头：中间变量

Local chain

Units Chain

Traps Chain

# APIs

- Value factories

- `v()` 是创建the instances的factory method。
- `RefType.v("java.lang.String")` 创建了一个字符串。
- `and IntConstant.v(5)` 创建整型常量5。

Type of the instance

value of the instance

# APIs: Scene

- 生成的应用由一个*Scene*对象代表。
  - *SootClass loadClassAndSupport(String className)*
    - Loads the given class and resolves all the classes necessary to support that class, that is, the transitive closure of all the references of classes within the class. （加载类及被该类引用的类。）
  - *void addClass(SootClass class)*
    - 把SootClass对象加入Scene.



# APIs: SootClass

- Scene中的具体的类由*SootClass*类表示。
  - *new SootClass(String name, int modifiers)*
    - 构造*SootClass*实例。
  - *void addField(SootField f)*
    - 为该类添加field。
  - *void addMethod(SootMethod m)*
    - 为该类添加method。

# APIs: SootField

- *SootField*类的实例代表Java。
  - *SootField(java.lang.String name, Type type, int modifiers)*
    - *SootField*类的构造函数。

# APIs: SootMethod

- *SootMethod*类的实例代表Java方法。
  - *SootMethod*(*java.lang.String* name,  
*java.util.List* paramTypes,  
*Type* returnType, *int* modifiers)
  - *void setName(String)*
  - *void setReturnType(Type t)*
  - *void setModifiers(int m)*
  - *void setActiveBody(Body b)*

# APIs: Body

- 方法体由*JimpleBody*类的实例代表。
  - *Chain getLocals()*
    - Returns a backed chain of the Locals in this method.
  - *Chain getTraps()*
    - Returns a backed chain of the Traps in this method.
  - *PatchingChain getUnits()*
    - Returns a backed patching chain of the Units (语句) in this method.
    - 在chain中添加statements (units)的方法见PatchingChain的API（下页ppt）

## API: PatchingChain

- *Class PatchingChain*  $< E$  extends *Unit*  $>$
- *boolean add(E o)*
  - Adds the given object to this Chain.
- *void addFirst(E u)*
  - Adds the given object at the beginning of the Chain.
- *void addLast(E u)*
  - Adds the given object at the end of the Chain.

# APIs: Local

- *newLocal(String name, Type t)*
  - *Jimple.v().newLocal("a", IntType.v())* 创建一个 *integer* 类型的、名为 *a* 的函数作用域的 Jimple variable。
- *void setName(String s)*
- *void setType(Type t)*

# APIs: Trap

- *Trap*类的实例代表异常处理结构。
- *newTrap(a, b, c, d)*
  - *Jimple.v().newTrap(a, b, c, d)* 创建一个Jimple trap（异常处理结构），其中被捕获的异常类型为 *d*，被监管的范围由 *a* 开始，由 *b* 结束，异常处理部分为 *c*，*a*、*b*、*c* 为语句标签。
- *void setBeginUnit(Unit u)*
- *void setEndUnit(Unit u)*
- *void setHandlerUnit(Unit u)*
- *void setException(SootClass c)*

# APIs: Unit

- The *Unit* interface
  - 代表instructions或statements，即3地址语句。
  - *AssignStmt newAssignStmt(Value lvalue, Value rvalue)*
    - 创建形如 *lvalue = rvalue*. 的语句。
  - *IdentityStmt newIdentityStmt(Value local, Value identityRef)*
    - 创建an identity statement of the form *local := identityRef*.
    - *local* is pre-loaded with meaning, such as *this* or the contents of parameters. ( *local* 具有预定义的形式以及含义。 )



- *InvokeStmt newInvokeStmt(InvokeExpr e)*
  - *InvokeExpr*的API见下一页ppt
- *ReturnVoidStmt newReturnVoidStmt( )*

## APIs: InvokeExpr

- InvokeExpr本身继承自Value类（后文介绍）。
- *void setArg(int index, Value arg)*
- *void setMethodRef(SootMethodRef smr)*
- *void setBase(Value base)*
  - 指定调用哪一个instance的方法。

# APIs: Type

- Base types:
  - *BooleanType.v()*, *ByteType.v()*, *CharType.v()*,  
*DoubleType.v()*, *FloatType.v()*, *IntType.v()*,  
*LongType.v()*, *ShortType.v()*, *VoidType.v()*
- *ArrayType.v(a, b)*
  - *a*是数组的元素的类型，*b*是维数。
- *RefType.v(a)*
  - *a*是被引用的类的名字。

# APIs: Modifier

- Modifiers are represented by final static integer constants (Modifiers can be merged together by "or"ing their values.):
  - *Modifier.ABSTRACT*
  - *Modifier.FINAL*
  - *Modifier.INTERFACE*
  - *Modifier.NATIVE*
  - *Modifier.PRIVATE*
  - *Modifier.PROTECTED*
  - *Modifier.PUBLIC*
  - *Modifier.STATIC*
  - *Modifier.SYNCHRONIZED*
  - *Modifier.TRANSIENT*
  - *Modifier.VOLATILE*
- Modifiers can be merged together by "or"ing their values.

# APIs: Value

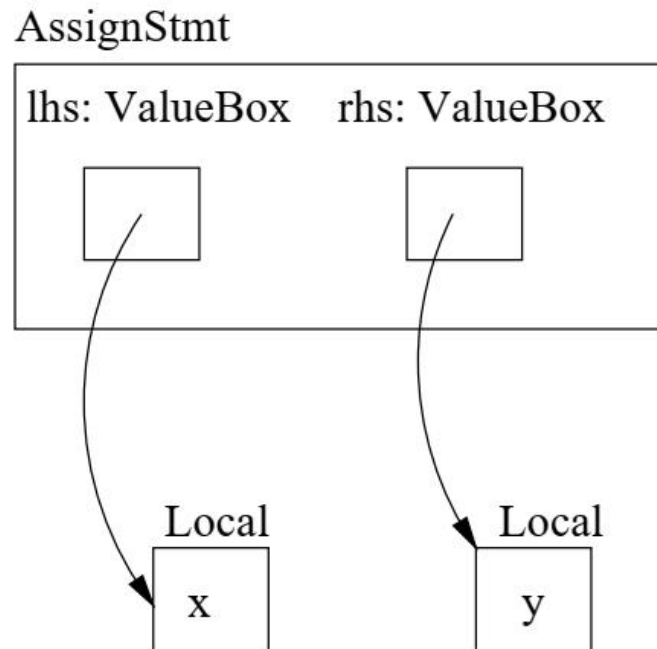
- 在JIMPLE中，*Values*包括*Constants*和*Expr*的子类等。
  - *newParameterRef(Type type, int n)*
    - 构造一个对指定类型的参数的引用，*n*指参数的序数，序数由0开始。
  - *newStaticFieldRef(SootField f)*
    - 构造一个对指定的field的static field reference。
  - *newVirtualInvokeExpr(Local base, SootMethod method, List args)*
    - 构造一个virtual invoke expression。
  - *newAddExpr(Value leftOp, Value rightOp)*
    - 构造一个加法表达式。

## APIs: Constants

- *IntConstant.v(a)*
- *FloatConstant.v(a)*
- *DoubleConstant.v(a)*
- *LongConstant.v(a)*
- *NullConstant.v()*
- *StringConstant.v(a)*

# APIs: Box

- 两类用于封装的Boxes:
  - 就是为了便于被引用。
  - *UnitBox*和*ValueBox*.



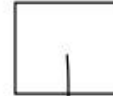
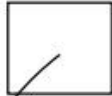
# APIs: Box

InvokeStmt

base: ValueBox

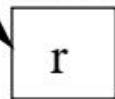
arg1: ValueBox

arg2: Value Box

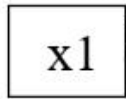


被调用方法所属的instance

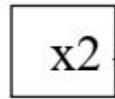
Local



Local



Local



- 便于优化



# Jimple 代码生成

- 参考群文件中的“编译原理实验报告（Jimple）”中的类CodeGenerator.
  - 该实验报告是其他学校的，他们的此法和语法分析是用ANTLR实现的，不是自己做的。
  - 他们在遍历ANTLR生成的语法树的过程中生成Jimple代码。

# 简单优先分析法(忽略)

- Bottom-up。
- 找句柄，采用最左归约。
- 用 $\lessdot$ 和 $\gtrdot$ 标识句柄的左、右边界。
  - 外 $\lessdot$ 内 $\sqsupset$ 内 $\sqsupset$ 内 $\gtrdot$ 外
- 算符优先分析法是简单优先分析法的一种特例。

例：有文法如下

$$(1) S \rightarrow AcBe$$

$$(2) A \rightarrow b$$

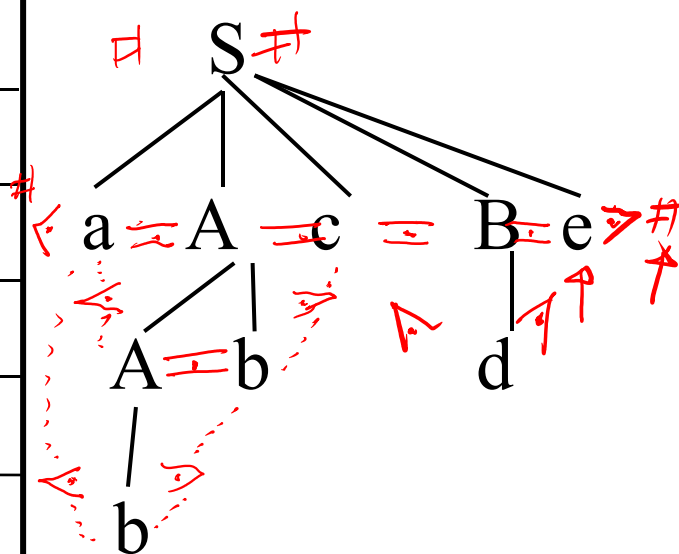
$$(3) A \rightarrow Ab$$

$$(4) B \rightarrow d$$

问：语句abbcde是不是该文法的合法语句？

| 步骤 | 栈       | 输入串      | 输出带     | 动作 |
|----|---------|----------|---------|----|
| 0  | #       | abbcde # |         |    |
| 1  | # a     | bbcde #  |         | 移进 |
| 2  | # ab    | bcde #   |         | 移进 |
| 3  | # aA    | bcde #   | 2       | 归约 |
| 4  | # aAb   | cde #    |         | 移进 |
| 5  | # aA    | cde #    | 2,3     | 归约 |
| 6  | # aAc   | de #     |         | 移进 |
| 7  | # aAcd  | e #      |         | 移进 |
| 8  | # aAcB  | e #      | 2,3,4   | 归约 |
| 9  | # aAcBe | #        |         | 移进 |
| 10 | # S     | #        | 2,3,4,1 | 归约 |
| 11 | 成功      |          |         |    |

$a \leftarrow A \rightarrow b \rightarrow c$   
 $a \leftarrow A$   
 $A \rightarrow a$   
 $b \rightarrow A$   
 $a / A / c$



# 简单优先分析方法的基本思想

- ① 对句型中相邻的文法符号规定优先关系，以寻找句型中的句柄。
- ② 规定句柄内各相邻符号之间具有相同的优先级。
- ③ 规定句柄两端符号优先级要比位于句柄之外而又和句柄相邻的符号的优先级高，以先归约句柄。
- ④ 对于文法中所有符号，只要它们可能在某个**最右推导**句型中相邻，就要为它们规定相应的优先关系，若某两个符号永远不可能相邻，则它们之间就无关系。

# 问题？

